

DSpark:基于置信度调度的推测性解码 半自回归生成

程鑫^{1,2,*}, 余兴凯^{2,*}, 邵晨泽^{2,*}, 李嘉实^{2,*}, 熊云帆^{2,*},
易谦², 朱嘉琪², 马世荣², 张小康², 叶家胜², 陈勤宇²,
邓承其, 余继平, 大麦戴², 张正艳², 魏一轩², 谭一轩²,
杨文凯², 徐润新², 于武², 徐哲安², 王轩宇², 陈牧阳²,
瑞天², 小毕², 郝哲文², 陈少元², 曹焕奇², 张辉帅¹, 赵东燕¹, 梁文峰², 1北, 张文涛²,
徐安义², 京大学² DeepSeek-AI { Chengxin, xingkai, shaochenze, js.li,
yunfanxiang}@deepseek.com

抽象的

推测性解码通过将草稿生成与目标验证解耦,加速了大型语言模型 (LLM) 推理。虽然近期的并行草稿生成器能够在单次前向传递中高效地生成较长的词元序列,但由于缺乏词元间的依赖关系,其接受度会迅速下降。此外,不加区分地验证这些扩展块会将宝贵的批处理容量浪费在拒绝风险高的词元上,从而严重降低高并发服务系统的吞吐量。我们提出了 DSpark,一个推测性解码框架,它将高吞吐量的并行生成与自适应的、负载感知的验证相结合。为了保持草稿质量,DSpark 采用半自回归架构 将并行主干与轻量级顺序模块相结合 引入块内依赖关系建模并缓解后缀衰减。为了优化系统效率,DSpark 采用置信度调度验证,根据估计的前缀存活概率和引擎特定的吞吐量特征,动态地为每个请求调整验证长度。在跨多个领域的离线基准测试中,DSpark 相比目前最先进的自回归和并行绘图器,显著提高了可接受的长度。当部署在 DeepSeek-V4 服务系统中,并在实际用户流量下运行时,DSpark 成功地减少了验证浪费。与已建立的生产基线 (MTP-1)相比,在吞吐量相同的情况下,DSpark 将每个用户的生成速度提高了 60%–85%。更重要的是,它通过防止在严格的交互约束下出现严重的吞吐量下降,实现了以前无法达到的性能层级,从而扩展了我们服务系统的帕累托前沿。为了促进社区发展,我们开源了[DSpark 检查点](#)。与 DeepSpec一起,一个用于推测性解码的算法驱动型训练库。

1. 引言

大型语言模型 (LLM)以自回归的方式生成文本:每个新词元都需要基于所有先前词元进行完整的正向传播,这使得推理延迟与输出长度成正比。由此导致的GPU利用率低和用户感知等待时间长,构成了……

*均等贡献。

生产环境中的语言学习模型 (LLM) 服务存在一个主要瓶颈,尤其是在对延迟敏感的场景 (例如实时对话助手和多轮代理工作流) 中。推测性解码 (Chen et al., 2023; Leviathan et al., 2023) 提供了一种有效的解决方案: 一个轻量级的草稿模型生成一个候选词块, 而完整尺寸的目标模型通过拒绝采样在一次前向传播中验证整个词块, 接受与目标分布一致的最长前缀, 并附加一个奖励词。由于验证是并行进行的, 并且接受规则能够精确地保持目标分布, 因此推测性解码能够在不损失任何质量的情况下加速生成过程。

草稿模型的设计决定了草稿延迟和接受率之间的权衡。早期的草稿算法是自回归的 (Cheng et al., 2024; Li et al., 2024b), 它们基于先前采样的代币来决定每个位置。然而, 它们的草稿延迟随区块大小线性增长, 迫使这些方法使用较短的区块和较浅的架构。为了突破这种顺序瓶颈, 并行草稿算法 (Cai et al., 2024; Chen et al., 2026; Liu et al., 2026a) 应运而生, 成为一种引人注目的替代方案: 所有草稿位置都在一次前向传递中生成, 使得草稿延迟几乎与区块大小无关。这种结构优势理论上使得并行草稿算法能够高效地生成更长的草稿区块。

然而, 充分发挥大型并行草稿块的潜力会带来两个关键瓶颈——一是生成质量, 二是系统效率。首先, 由于并行草稿生成器独立预测每个位置, 因此无法对块内令牌间的依赖关系进行建模。这种独立性会导致多模态冲突, 并在后续位置迅速降低接受率 (Gu et al., 2018; Huang et al., 2022b)。其次, 确定最佳验证长度仍然是一个挑战。虽然并行生成可以轻松生成较长的草稿块, 但对所有提议的令牌进行不加区分的验证会降低系统吞吐量, 尤其是在高并发工作负载下 (Hu et al., 2026; Liu et al., 2024c)。理想的验证长度取决于两个方面。在数据方面, 结构化请求 (例如代码) 的接受率自然高于开放式聊天 (Abramovich et al., 2026; Xia et al., 2024)。在系统方面, 在轻负载下验证额外的令牌几乎不消耗任何资源。然而, 在高负载情况下, 验证具有高拒绝风险的令牌会占用关键的批处理容量, 而这些容量原本可以用于服务其他活跃请求 (Liu 等人, 2024b; Wu 等人, 2025)。

为了解决这些瓶颈问题, 我们引入了 DSpark, 这是一个推测性解码框架, 它将高吞吐量并行生成与自适应、负载感知验证相结合。DSpark 的核心设计是通过两种互补机制来解决草稿生成和验证中固有的权衡取舍。

首先, 为了克服词元间依赖关系缺失的问题, DSpark 采用了半自回归架构。它保持计算量巨大的草稿主干完全并行, 仅附加一个轻量级的串行输出头来注入本地转换信息。这种设计既保持了并行模型的草稿速度, 又显著缓解了后缀衰减问题。

其次, 为了解决系统级瓶颈, DSpark 采用了置信度调度验证。DSpark 将置信度头 (用于估计每个位置前缀的存活概率) 与硬件感知调度器相结合, 从而动态地为每个请求调整验证时长。该调度器利用实时引擎的吞吐量曲线, 将目标验证预算仅分配给预期收益最高的令牌。

我们对 DSpark 在受控离线基准测试和生产规模的在线部署中进行了广泛的评估。在涵盖数学推理、代码生成和日常聊天等受控离线基准测试中, DSpark 的性能始终优于强大的基准测试。具体而言,

在 Qwen3-4B、8B 和 14B 目标模型中 (Yang 等人, 2025), 它改善了宏观平均值。与自回归 Eagle3 模型 (Li 等人, 2026b) 相比, 可接受的长度分别提高了 30.9%、26.7% 和 30.0%。与并行 DFlash (Chen 等人, 2026) 相比, 分别提高了 16.3%、18.4% 和 18.3%。除了这些主要指标之外, 我们精细的位置分析还揭示了不同的生成特征。

不同绘图员的经验, 通过实证研究证明了 DSpark 如何成功地结合了高水平的绘图员具有自回归模型后缀一致性的平行模型的初始标记容量。

除了离线评估之外, 我们还在 DeepSeek-V4 (DeepSeek-AI, 2026) 中部署了 DSpark。服务系统用于评估其在真实用户流量下的性能。与之前的 MTP-1 相比。生产基线 (DeepSeek-AI, 2024), DSpark 显著扩展了系统的运行范围。封装。具体来说, 它能持续将每个用户的生成速度提高 60%–85% (V4-Flash) 和 57%–78% (V4-Pro), 总吞吐量容量匹配。此外, 严格的服务级别协议 (SLA), 其中基线容量严重下降。例如 Flash 的 TPS 为 120, Pro 的 TPS 为 50。DSpark 通过减少验证开销来维持强大的吞吐量。通过克服这一性能瓶颈, DSpark 实现了严格的交互性。以前无法达到的层级, 有效地改变了 LLM 服务的帕累托前沿。

为了促进开源社区的集体进步, 我们正在努力……公开可用的工件。具体来说, 我们发布了训练好的 DSpark 检查点。对于两者 DeepSeek-V4-Flash (预览版) 和 DeepSeek-V4-Pro (预览版) 型号。此外, 我们开源的 DeepSpec, 一个算法驱动的训练库, 包括 Eagle3、DFlash 和 DSpark。这些成果旨在支持对高效 LLM 服务的进一步研究。

2. 背景

2.1 推测性解码

自回归语言模型每次前向传播生成一个词元, 因此推理延迟较高。与输出长度成正比。推测性解码 (Chen et al., 2023; Ge et al., 2022; Leviathan 等, 2023) 利用轻量级草稿模型加速目标模型的推断。

在每个解码周期中, 草稿模型都会提出候选词元。目标模型在 1, …… , 在一次前向传递中验证所有候选对象, 接受与其一致的最长前缀。自有发行。

具体来说, 在每个选秀位置, 目标模型计算自身的分布并将其与选秀分布进行比较。代币以概率被接受。

$\min(1, p_i / q_i)$ 。验证从左到右进行: 位置 1 处的第一个拒绝结果为丢弃所有后续代币, 无论其质量如何。+1, ……

令 n 表示每个周期内被接受的令牌数量, 令 d_{draft} 和 v_{verify} 为实际时钟时间。分别计算草稿和验证过程的时间。每次生成的平均延迟。令牌是:

$$= \frac{\text{草稿} + \text{核实}}{n} \quad (1)$$

因此, 提高速度可简化为三个杠杆: 降低草稿长度 (加快草稿速度)、提高草稿长度 (提高草稿质量) 或减少有效验证时间 (更智能地验证)。

2.2. 绘图员架构

草案模型的设计决定了草案的制定和权衡方式。现有方法存在缺陷。分为两类。

自回归选秀器。自回归选秀器按顺序生成选秀标记,每个位置都以先前采样的标记为条件 (DeepSeek-AI, 2024; Li 等人, 2024b,c, 2026b; Zhang 等人, 2025)。这种明确的依赖关系赋予了强大的建模能力,但草拟工作却存在问题。

成本随区块大小线性增长:草案 $\propto$,这迫使自回归制图者使用较小的区块。

以及浅层结构以保持较低的草稿高度。为了弥补短草稿,采用了基于树状结构的草稿设计。验证 (Miao 等人, 2024)将候选路径扩展成树状结构,并通过该树验证多条路径。虽然需要注意,但大量的验证令牌会降低整体服务吞吐量。

并行选牌器。并行选牌器在一次前向传递中生成所有选牌标记,从而

草稿大小几乎与区块大小无关 (Cai et al., 2024; Chen et al., 2026; Li et al., 2025a; Liu)。

等, 2026a; Sandler 等, 2026)。这使得可以创建更大的块 (例如, $=16$),而无需延迟成比例增加。

其中,DFlash (Chen et al., 2026)是一款先进的并行绘图软件,它满足了……其基于从目标模型 (KV注入)提取的丰富上下文特征的草图模型。预填充时,将来自一组目标层 $\{1, \dots\}$ 的隐藏状态连接起来并投影到草稿隐藏空间:

$$\text{ctx} = \text{RMSNorm} [\dots; \dots; \dots], \quad (2)$$

其中 $\in R$ 表示共享投影。这些上下文特征会被注入到每个草稿中。

通过沿序列维度将它们与草稿块表示连接起来形成图层键和值:

$$= [\dots; \text{ctx}; \dots], \quad = [\dots; \text{ctx}; \dots] \quad (3)$$

一个模块内的所有位置都相互双向关注,并与注入的目标上下文进行双向关注。

该草稿模型与目标模型共享嵌入层和语言建模头。

(两者均已冻结)。它以锚点标记1的嵌入向量和掩码到词库的嵌入向量作为输入,并在一次前向传播中生成所有掩码位置的 logits。由于

选秀只需要一次向前传球,无论阻挡大小如何,DFlash 可以承受更深的传球。

在相同的延迟预算下,架构和更大的模块比自回归绘图器更大。

3. 建筑

DSpark 的概览如图1所示。回顾公式 1,每个令牌的延迟为

推测性解码的计算公式为:(草稿+验证) / 。自回归式草稿者虽然能取得高成就,但代价也很大。

选秀 $\propto$;并行选秀者将选秀简化为单次选秀,但会做出牺牲,因为每个位置都是

独立预测。同时,固定长度的验证会浪费低置信度的验证。

后缀标记几乎肯定会被拒绝。DSpark 通过两种方式解决了这些限制。

互补成分:

·半自回归生成 (第 3.1 节)。并行主干处理大部分草稿。

计算使得草案几乎与……无关。

然后是一个轻量级的顺序块

在草稿代币之间注入依赖关系,以最小的额外延迟实现改进。

·置信度计划验证 (第 3.2 节)。置信度负责人估计每个位置

接受概率,硬件感知调度器利用这些估计值进行剪枝。

低置信度后缀标记,减少不必要的验证计算。

本文中,我们交替使用“锚定代币”和“奖励代币”这两个术语来指代最终生成的代币。由上一轮解码的目标模型决定。

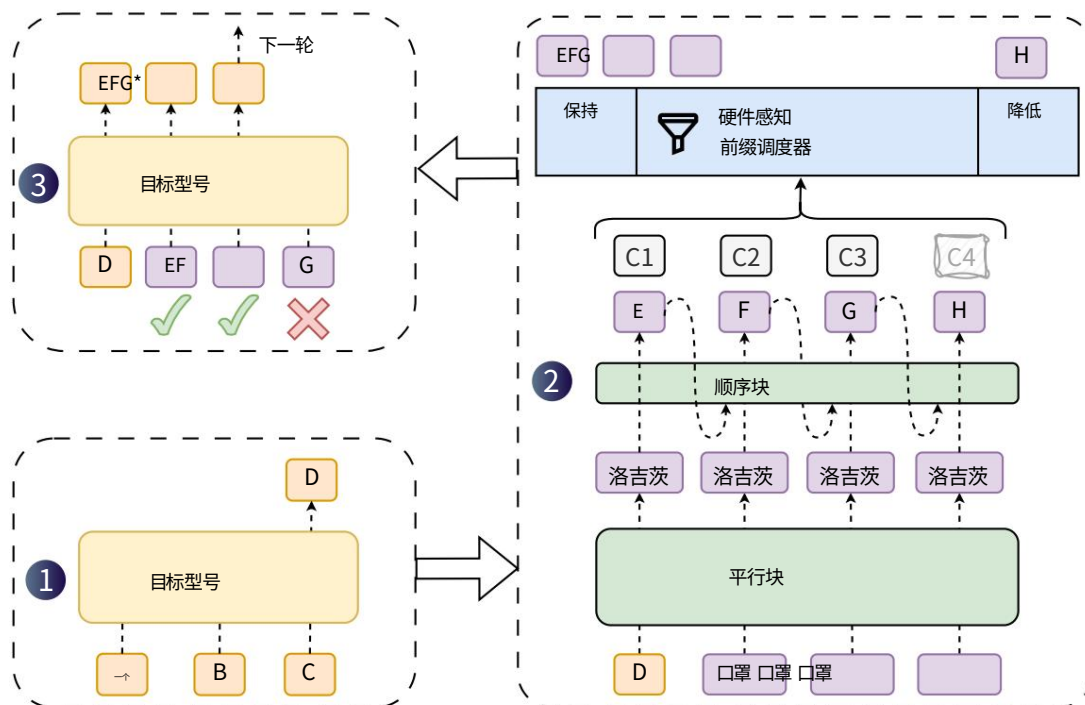


图 1 | DSpark 架构和解码周期。给定提示词ABC,目标模型执行一个步骤来生成下一个词D,即草稿阶段。DSpark 使用D, 这作为输入,采用强大的并行骨干网络和

轻量级顺序头部,用于生成草稿代币EFGH及其对应的代币置信度得分1-4。硬件感知前缀调度器随后评估这些得分。

保留前缀EFG并丢弃低置信度标记H。最后,目标模型验证并行执行预定前缀。如图所示, E和F被接受,而G被拒绝。

提示模型生成修正后的令牌G以完成当前轮次。

这两个组件结合起来,可以让 DSpark 更好地进行草稿生成和更智能的验证。我们详细介绍每个组件。以下。

3.1 半自回归生成

并行选器在一次前向传递中生成所有选秀逻辑值,因此每个预测都不能对区块中其他位置采样的标记进行条件判断。当上下文允许多个合理的例如,“当然”和“没问题”这样的延续性词语,如果使用平行起草,可能会导致语句不连贯。

诸如“有问题”或“当然”之类的组合,因为每个立场都会边缘化超过考虑所有可能的先行者,而不是以实际抽样的先行者为条件 (Gu et al., 2018; Huang等人 (2022a)指出,因此,接受率沿区块迅速下降,浪费了选材。

并进行验证计算。因此,我们采用半自回归结构,将草稿拆分为多个部分。生成分为两个阶段:

并行阶段。并行主干网 (在我们的实例中,是 DFlash (Chen et al., 2026))运行着一个对整个块进行一次前向传播,生成隐藏状态 $1, \dots$, 和基础logits。

1, ..., 我们仅对原有的 DFlash 主干网进行了微小的修改:而不是通过输入锚定标记和掩码标记,并仅预测掩码位置,我们处理

锚点本身作为第一个预测位置,因此输入标记(锚点+ -1 掩码)生成草稿逻辑值。这在保持草稿质量相近的情况下减少了草稿计算量。

顺序阶段。顺序阶段在基础logits的基础上,增加了一个与前缀相关的logits。
转换偏差(0, <,) ,允许每个选秀位置根据先前采样的情况进行调整
区块内的代币。与其定义一个全局归一化的能量模型,不如采用顺序方式。
该阶段通过自回归因子分解诱导因果块分布:

$$p(z_t | z_{<t}) = \prod_{v=1}^V p(z_t | z_{<t}, v) = \frac{\exp(\langle z_t, v \rangle + \langle z_{<t}, v \rangle)}{\sum_{v' \in V} \exp(\langle z_t, v' \rangle + \langle z_{<t}, v' \rangle)} \quad (4)$$

这里, z_t 表示上一个验证周期中的锚标记,是基础logit向量。
由位置处的并行骨干网络生成, V 是词汇表。在推理时,
根据(· | $z_{<t}$) 从左到右依次对块进行采样。因为这种采样过程
由于本质上是顺序执行的,因此该模块必须在计算上是轻量级的(顺序执行≪并行执行)。
因此,整体草稿延迟仍然主要由并行阶段决定。我们描述了两种情况。
下面顺序块的实例化。

·马尔可夫头。最简单的实例化限制其仅依赖于直接

前一个标记,将其简化为一阶转换(−1,)。原则上,这是一个

完整的 $\times$ 矩阵;我们用低秩分解来近似它=其中

$U \in \mathbb{R}^{1 \times V}$ 和 $V \in \mathbb{R}^{V \times 1}$ 给定前面的标记

是:

$$(U, V) = \frac{1}{\sqrt{2}} [U_1, U_2] \in \mathbb{R}^{1 \times V}, \quad (5)$$

其中 U_1 为逻辑回归投影,低秩

因式分解(默认值为 256)既能保持较小的存储空间,又能减少每一步的计算量,从而使
即使对于大型词汇表,顺序循环也很高效。回到之前的例子:

一旦位置 1 采样到“of”,马尔可夫头就会提升“course”并抑制“problem”。

在位置 2,可以减轻跨模式碰撞。

· RNN 头部。马尔可夫头部在单步之后是不具有记忆性的 位置无法访问。

在 RNN 头部之前,令牌通过维护一个循环状态来放宽这种限制。

模块会在一个代码块内累积完整的前缀历史记录。在每个步骤中,模块都会连接以下内容:

当前状态 $z_t \in \mathbb{R}$ 和主干,前一个词嵌入 $U_{t-1} \in \mathbb{R}$,

将隐藏的 $z_t \in \mathbb{R}$ 转换为输入向量 $U_t = [-1; 1 - z_t]$ $\in \mathbb{R}^2$ +门控更新: ,然后应用单个

$$U_t = (U_{t-1} \odot -1 + 1 - z_t) \odot \tanh(\tanh(\langle z_{<t}, \cdot \rangle)), \quad (6)$$

其中分, , $\in \mathbb{R}^{(2+1) \times V}$ 由单个线性投影联合参数化,该线性投影是
为门电路、候选电路和输出电路三个部分。状态 z_0 初始化为零。

3.2. 置信度计划验证

半自回归架构使 DSpark 能够高效地生成大型草稿块。

然而,生产更多草稿代币并不一定意味着更高的端到端性能。

提高速度。不加区分地验证整个草稿块实际上会降低整个系统的性能。

吞吐量,尤其是在高并发场景中 (Hu et al., 2026; Liu et al., 2024c) 。

这种性能瓶颈源于两个相互作用的因素。首先,在数据方面,草稿的接受率在不同领域之间存在固有差异:结构化文本(例如代码)的接受率自然较高,而开放式聊天记录接受率则显著较低(Abramovich et al., 2026; Xia et al., 2024)。其次,在系统方面,验证额外令牌的实际成本严格取决于引擎负载。在系统负载较低的情况下,即使验证被拒绝,额外的验证也只会带来极小的损失。然而,在高并发部署中,每一次不必要的验证都会占用目标模型的批处理容量,而这些容量原本可以用于处理其他活跃请求(Liu et al., 2024b; Wu et al., 2025)。

因此,要充分发挥大型草稿区块的潜力,需要一个统一的机制,将目标模型计算资源仅分配给预期收益为正的令牌。DSpark通过将预测前缀存活概率的置信度预测器(3.2.1节)与硬件感知的前缀调度器(3.2.2节)相结合来实现这一点,后者能够根据当前系统负载动态确定最佳验证长度。

3.2.1. 信心主管

借鉴Huang等人(2024)和Wang等人(2026)的研究,置信度头为每个草稿位置输出一个标量估计值 $\in (0, 1)$ 。关键在于,它模拟了在区块中所有前面的令牌均已被接受的情况下,位置为 i 的草稿令牌通过目标验证的条件概率。该架构采用轻量级线性投影,后接一个sigmoid函数:

$$h_i = [h_{i-1}; 1[-1]], \quad (7)$$

其中 h_{i-1} 是骨干网络的隐藏状态, $1[-1]$ 是来自前一个草稿标记的马尔可夫嵌入。我们使用解析接受率进行监督,每一步的接受率由草稿分布和目标准分布之间的总变异距离决定:

$$s_i^* = 1 - \frac{\|p_i - q\|}{\|p_i - q\| + 1}. \quad (8)$$

事后校准。与仅需置信度分数即可正确排序草稿令牌质量的基于阈值的验证启发式方法(Huang et al., 2024; Li et al., 2024b; Zhang et al., 2026b)不同,我们提出的硬件感知调度方法(详见3.2.2节)精确地需要累积接受概率的绝对值来计算预期接受长度。由于神经网络置信度估计通常过于自信(Guo et al., 2017; Ovadia et al., 2019),直接使用原始置信度分数会扭曲吞吐量估计,导致调度结果不理想。

为了解决这个问题,我们引入了序贯温度缩放(STS)。由于每个模型都模拟了一个条件概率,链式法则表明,草稿前缀被接受的联合概率可以分解为累积乘积

使用预留的验证集,STS从左到右依次校准该联合概率。具体来说,在每个位置 $i \in \{1, \dots, L\}$,我们执行简单的1D网格搜索,以找到使累积乘积的预期校准误差(ECE)最小化的最优温度标量(Naeini等人, 2015),同时保持所有先前位置的已校准分数不变。至关重要的是,温度缩放是一种保序变换:它修正预测概率以匹配经验接受率,而不会破坏置信度评估者学习到的相对草稿代币排名。

3.2.2. 硬件感知前缀调度器

算法 1硬件感知前缀调度器

要求:活跃请求 $\in \{1, \dots, \}$;置信序列, $1, \dots$,阶跃曲线SPS(ℓ)

根据要求;已建立个人资料

确保:已选择每个请求的前缀长度 $\ell_1^*, \dots, \ell_n^*$

- 1: for = 1 执行
- 2: 计算前缀生存概率: $\leftarrow$, 3: 结束循环
- 4: 构建候选空间 $E \leftarrow \{(\ell, \theta) \mid \theta > 0\}$ 并按降序排序, 5: 初始化状态: $\ell \leftarrow 0$ (对于所有), 批次大小 $\leftarrow$; 预期接受数 6: 初始化跟踪: $\Theta_{\text{best}} \leftarrow \cdot \text{SPS}(\ell)$; 选定长度 $\ell^* \leftarrow$
- 7: 对于每个 $(\ell, \theta) \in E$ 按排序顺序执行以下操作
- 8: $\ell \leftarrow$; $\leftarrow + 1$; 9: 当前吞吐 $\leftarrow \cdot +$
- 量 $\Theta \leftarrow$ 如果 $\Theta > \Theta_{\text{best}}$ 则 $\cdot \cdot \text{SPS}(\ell)$
- 10:
- 11: $\Theta_{\text{best}} \leftarrow \Theta$; 更新选定长度 $\ell^* \leftarrow \ell$
- 12: 否则
- 13: 休息
- 14: 结束 if
- 15: 结束
- 16: 返回 $(\ell_1^*, \dots, \ell_n^*)$ 达到 Θ_{best}

先前的方法 (Huang et al., 2024; Li et al., 2024b) 通常采用静态阈值。置信度评分用于确定验证时长。虽然在孤立的、单次请求的情况下有效,在假设条件下,静态阈值在高并发生产系统中可能并非最优选择。验证草稿令牌的效用很大程度上取决于当前的系统负载。

为了解决这个问题,我们将验证长度选择建模为全局吞吐量最大化问题。

问题 (算法1)。考虑一批活跃请求。对于请求 i , 令 ℓ_i 表示每个位置的置信度估计, 令 $\ell \in \{0, \dots, \}$ 表示已安排的验证。

长度。因为推测性解码动态地只接受草稿标记作为连续的

前缀, 位置处的令牌的生存概率是累积乘积,

在单个验证步骤中, 发送到目标的总批次大小 (以令牌为单位衡量) 模型是 $\sum_{\ell=1}^{\ell} (1 + \ell)$, 预期成功接受的代币数量为 $\sum_{\ell=1}^{\ell} \ell \cdot \text{SPS}(\ell)$ 表示引擎吞吐量, 以每秒步数衡量, 对于给定前向传递批次大小。至关重要的是, 该容量曲线在引擎运行期间仅绘制一次。初始化并存储为轻量级成本表。我们的调度器随后旨在最大化通过动态选择验证方式, 预期系统范围令牌吞吐量 $\Theta = \cdot \text{SPS}(\ell)$ 。

长度 $\ell_1, \dots, \ell_n$ 。

尽管寻找 Θ 的全局最大值看起来像是一个组合搜索问题, 但目标结构允许采用高效的贪婪解法。因为关于 ℓ (即 -1) 单调非增, 所以预期接受代币的边际收益为 $\leq$

将请求的验证长度从 -1 扩展到正好是。这种单调性确保通过全局排序候选令牌, 自然会尊重区块内的前缀依赖关系。因此, 如果总验证批次大小固定, 则最优分配 $\{\ell\}$ 将通过贪婪地选择存活概率最高的草稿代币来确定所有 $\{ \}$ 。

基于这一见解, 可以沿着这种贪婪的录取路径评估优化效果。

我们首先对所有有效的前缀扩展按生存概率降序排列。为了动态确定最佳目标批次大小,我们从排序后的池中逐步添加令牌,并通过 (1) 从预先分析的成本表中查找来更新预期吞吐量 Θ 。

无损推测解码严格要求非预测性:准入决定不能依赖于未来的候选标记 (Chen et al., 2023; Leviathan et al., 2023)。

由于我们的置信度依赖于先前采样标记的马尔可夫特征,因此计算下一个生存概率+1明确需要实例化的候选者。回顾性全局搜索因此会无意中泄露到步骤的准入决策中,从而引入选择偏差(我们在附录A中提供了一个具体的反例来证明这种理论违背)。

为了确保严格的因果关系,调度器 (算法1)采用了一种提前停止机制。当吞吐量下降 ($\Theta \leq \Theta_{\text{best}}$)时,立即停止贪婪搜索,截断决策仅依赖于截至该步骤已处理的前缀。这使得准入事件与后续令牌隔离,从而确保精确的目标分布恢复。需要注意的是,这种分步提前停止机制只有在目标函数 Θ 为单峰分布时才能获得全局最大吞吐量,这隐含地假设硬件容量曲线平滑衰减。我们将在 5.2 节中讨论针对实际应用中非平滑的SPS特性和异步系统流水线所需的工程调整。

3.3 培训

在训练过程中,我们从每个目标序列中随机抽取多个锚框位置,形成 -token 块作为训练数据。目标模型在整个训练过程中保持冻结状态;草稿模型共享其嵌入层和语言建模头,并保持冻结状态,仅更新骨干草稿器、序列块和置信度头。

训练目标由三部分组成:交叉熵损失 L_{ce} 、分布-匹配损失 L_{tv} 和置信度损失 L_{conf} 。这三个损失均按 $\exp(-(i-1)/\alpha)$ 进行位置加权 (Chen 等人, 2026), 这强调了在基于前缀的验证下对预期接受长度贡献更大的早期区块位置。交叉熵损失 L_{ce} 用于训练起草者预测正确的下一个标记:

$$L_{\text{ce}} = - \sum_{i=1}^N \log p(y_i | \mathbf{x}_{1:i-1}), \quad (9)$$

其中 y_i 是真实标记, p 是草稿分布。分布匹配损失 L_{tv} 惩罚草稿分布和目标分布之间的总变异距离:

$$L_{\text{tv}} = \sum_{i=1}^N \left| \frac{p(y_i)}{p(y_i) + q(y_i)} - \frac{q(y_i)}{p(y_i) + q(y_i)} \right|. \quad (10)$$

由于总变异距离是接受率的直接代理:每步接受率 $\frac{p(y_i)}{p(y_i) + q(y_i)} \approx \frac{1}{2}$ (Leviathan et al., 2023),因此最小化 L_{tv} 可以直接概率等于 $1 - \text{预期接受率}$ 最大化。置信度损失 L_{conf} 是一个二元交叉熵,用于训练置信度头,以根据公式8预测软接受标签:

$$L_{\text{conf}} = - \sum_{i=1}^N \left[\log p(\hat{y}_i | \mathbf{x}_{1:i-1}) + (1 - \hat{y}_i) \log(1 - p(\hat{y}_i | \mathbf{x}_{1:i-1})) \right]. \quad (11)$$

总体目标是三个项的加权组合（默认权重分别为 $ce = 0.1$, $tv = 1.0$ ） $\therefore 0.9$, $conf$

电视

$$L = ce L_{ce} + tv L_{tv} + conf L_{conf} \quad (12)$$

4. 实验

本节使用离线基准测试验证 DSpark 的草稿质量,并在第 5 节报告置信调度器在在线生产流量下的有效性。实验设置在第 4.1 节中描述,主要结果在第 4.2 节中描述,其他分析在第 4.3 节中描述。

4.1 实验装置

目标模型和草稿模型。我们使用四个目标模型评估 DSpark,这些模型涵盖不同的尺度和模型族:Qwen3-{4B, 8B, 14B} (Yang 等人, 2025)和 Gemma4-12B (Google DeepMind, 2026)。对于草稿模型,我们将 DSpark 与两个具有代表性的草稿生成器进行比较:DFlash (Chen 等人, 2026),一种最先进的并行草稿生成器;以及 Eagle3 (Li 等人, 2026b),一种基于训练时测试 (TTT)的自回归草稿生成器。为了公平比较,我们使用相同的训练框架重新训练所有草稿生成器。并且基于相同的数据。我们将 Eagle3 的 TTT 范围 (7)与 DFlash 和 DSpark 使用的块大小 (7) 对齐,并且所有绘图工具都使用相同的目标模型特征层。对于绘图模型层数,我们为 Eagle3 设置为 1,为 DSpark 和 DFlash 设置为 5 (Chen 等人, 2026)。除非另有说明,DSpark 指的是马尔可夫头变体;我们将在 4.3.2 节中研究 RNN 头变体。

训练数据。我们使用开源的 PerfectBlend 数据集 Open-PerfectBlend (Xu 等人, 2024)²,该数据集包含 130 万个样本。它是一个通用指令数据集,包含聊天 (17.6%)、数学 (39.4%)、代码 (38.9%)和指令遵循数据 (4.1%)。我们仅使用 Open-PerfectBlend 中的提示;每个目标模型均使用推荐的采样参数重新生成响应。每个模型均训练 10 个 epoch 以确保完全收敛。数据生成和评估均采用非思考模式。

评估协议。我们评估不同算法在三个领域的性能:

1. 数学推理,包括 GSM8K (Cobbe 等人, 2021)、MATH500 (Lightman 等人, 2024)和 AIME25 (Zhang 和 Math-AI, 2025)。
2. 代码生成,包括 MBPP (Austin 等人, 2021b)、HumanEval (Chen 等人, 2021)和 LiveCodeBench (Jain 等人, 2025)。
3. Daily Chat,包括 MT-Bench (Zheng 等人, 2023)、Alpaca (Taori 等人, 2023)和 Arena-Hard (Li 等人, 2024a, 2025b)。

对于所有基准测试,我们使用标准推测解码 (Chen et al., 2023; Leviathan et al., 2023),并将采样温度设置为 1.0。我们报告每轮解码的接受长度 ()。对于所有代码草拟器,我们使用基于链的代码草拟。

²<https://huggingface.co/datasets/mlabonne/open-perfectblend> 3. 为明确起见,除非另有说明,所有报告的接受长度和接受率指标均包含目标生成的奖励代币。

表 1 |主要推测性解码结果。我们报告每次解码的可接受长度 () 。
针对不同的目标模型和领域,取整 (数值越高越好) 。粗体字标示最佳结果。

目标	绘图员	数学			代码			聊天		
		GSM8K	MATH	AIME25	MBPP	HumanEval	LCB	MT-Bench	Alpaca	Arena-Hard
Qwen3-4B	鹰3	5.14	4.62	3.92	3.69	4.16	3.77	2.39	2.26	2.55
	DFlash	5.40	4.85	4.15	4.40	4.74	4.18	3.07	2.96	2.83
	DSpark 6.11 Eagle3	5.70	4.89	5.13	5.38	4.86	3.64	3.54	3.29	
Qwen3-8B	DFlash	5.30	4.77	3.91	3.96	4.33	4.17	2.66	2.54	2.54
		5.33	4.91	4.07	4.36	4.64	4.39	3.11	2.98	2.81
	DSpark 6.17 Eagle3	5.78	5.01	5.16	5.52	5.17	3.72	3.58	3.21	
Qwen3-14B	DFlash	5.24	4.60	3.71	3.81	4.14	4.01	2.62	2.47	2.48
		5.41	4.84	3.98	4.44	4.59	4.33	3.10	2.94	2.72
	DSpark 6.21 Eagle3	5.74	4.94	5.26	5.43	5.02	3.70	3.58	3.13	
Gemma4-12B	DFlash	5.87	5.46	4.83	4.72	5.37	4.16	3.19	3.06	2.72
	5.45 DSpark 6.05		5.04	4.22	4.39	4.95	3.70	2.98	2.84	2.59
			5.78	5.12	5.11	5.64	4.51	3.49	3.35	2.92

4.2 实验结果

为了将原始草稿质量与系统级调度策略区分开来,我们进行了离线评估。
禁用置信度调度器,强制所有起草者提议固定数量的代币。
表1报告了主要结果,以每轮平均接受长度 () 衡量。
DSpark 的性能始终优于自回归基线模型 (Eagle3)和并行模型。
在所有评估的目标模型和基准领域中,均采用基线 (DFlash) 。具体而言,
在 Qwen3-4B、8B 和 14B 型号中,DSpark 提高了宏平均可接受长度。
与 Eagle3 相比,DSpark 的性能分别提升了 30.9%、26.7% 和 30.0%。同样,与 DFlash 相比,DSpark 的性能也提升了 30.0%。
在三个维度上,相对改进幅度分别为 16.3%、18.4% 和 18.3%。至关重要的是,
优势在各个车型系列中都具有普遍性,这一点已由其一致的性能所证明。
达到 Gemma4-12B 目标价位。

除了平均改进之外,表 1 还揭示了一个显著的领域效应:已接受的领域
结构化任务的完成时间自然会更长 (例如,Qwen3-4B 的数学题为 5.57 分钟,编程题为 5.12 分钟) 。
比开放式聊天 (3.49)更准确。这种数据可预测性的固有差异意味着静态
验证长度通常会将计算资源浪费在极有可能被拒绝的尾随令牌上。
这直接促使我们采用置信度调度验证,它可以动态地修剪
根据预期接受度起草草稿。

4.3 实验分析

4.3.1. 为什么并行生成可以优于自回归？

表 1 呈现了一个反直觉的观察结果:并行绘图器 (DFlash) 和半自回归绘图器 (DSpark) 通常比完全自回归绘图器产生更长的可接受长度。
绘图员 (Eagle3) 。这一发现与通常的预期相反,即逐步自回归模型比并行模型产生更高质量的序列 (Israel 等人, 2026; Ren 等人, 2020;郑等人, 2025) 。

为了分析这种行为,我们考察了宏观层面可接受长度之外的性能。
使用 Qwen3-4B 目标模型和第 4.1 节中描述的基准数据集,我们引入
在实际的推测性解码推广过程中,跟踪了按位置划分的条件接受情况。具体来说,对于给定的草稿位置,评估分母仅统计以下情况:



图 2 | 位置条件接受率。我们报告了每个选词位置的经验条件接受率,该接受率是使用 Qwen3-4B 目标模型在每个领域内对基准测试结果取平均值得到的。与标准前缀存活率不同,该指标通过消除先前拒绝的惩罚,分离出每个位置的基线预测质量。请注意,自回归选词器 (Eagle3)保持稳定或呈上升趋势,而并行选词器 (DFlash)则存在后缀衰减现象。

目标模型成功验证并接受了从 1 到 -1 的所有先前草稿标记。该指标随后计算这些有效实例中,位置 1 的标记也被接受的比例。这种方法确保位置评估不会受到先前前缀错误的影响,从而揭示每个特定步骤的潜在预测质量。图 2 详细展示了这些测量结果,并清晰地展示了不同架构之间的行为差异。

位置 1 的容量优势。在第一个轮抽位置,两种架构都仅基于目标上下文预测下一个词。此处的性能差异完全源于架构容量:像 Eagle3 这样的自回归模型由于其延迟 () 而只能构建较浅的网络,而 (1) 并行轮抽器则可以构建更深的网络。这种结构性差异在位置 1 产生了显著的准确率优势,DFlash 的初始准确率明显高于 Eagle3 (例如,在 Math 上为 0.88 对 0.81,在 Chat 上为 0.72 对 0.53)。

由于推测性解码采用严格的前缀匹配生存机制,因此第一个代币具有最高的杠杆作用。如果该代币被拒绝,则整个区块将立即失效。

因此,这种初始容量优势不成比例地提高了最终接受的长度,解释了为什么尽管在后期位置接受度迅速下降,但平行起草者最终在全球范围内的表现仍然优于自回归起草者。

后段位置独立性的局限性。考察曲线的尾部 (位置 2 至 7) 可以发现独立并行生成的固有局限性。随着早期词元锁定特定的语义路径,后续词元自然变得更加可预测。

像 Eagle3 这样的自回归模型能够有效地利用这种条件确定性,在区块深处维持甚至提高条件接受度 (例如,在 Chat 区块中从 0.53 提高到 0.74)。相比之下,DFlash 的接受度衰减速度很快,在 Code 区块中从 0.87 下降到 0.78,在 Chat 区块中从 0.72 下降到 0.63。由于每个平行位置都会对所有可能的先前标记进行边缘化,而不是基于精确采样的前缀,因此该模型经常提出不一致的后缀组合。这种模式被称为多模态冲突 (Gu 等人, 2018; Stern 等人, 2018)。

利用半自回归缓解后缀衰减。前面的分析突出了一个明确的架构目标:将并行骨干网络的高容量用于初始词元,并与自回归模型的依赖性建模用于后续词元相结合。这直接

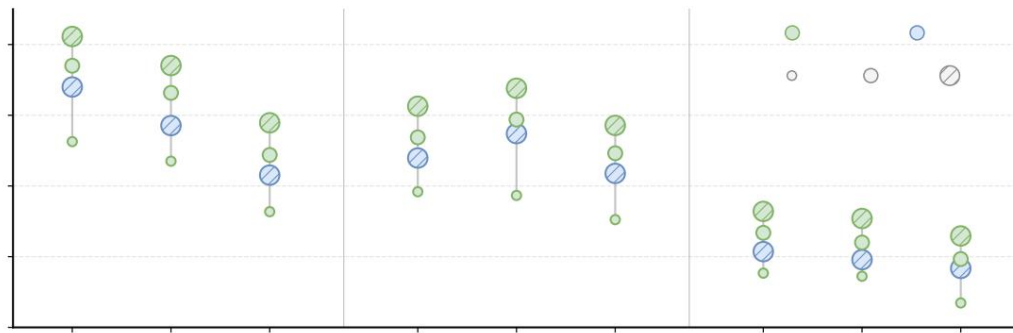


图 3 | 绘图深度的影响。在方案长度固定的情况下,随着绘图层数的增加,DSpark 的性能得到提升。值得注意的是,即使是较浅的 2 层 DSpark 也优于较深的 5 层 DFlash 基线模型,这凸显了顺序建模的参数效率优势。

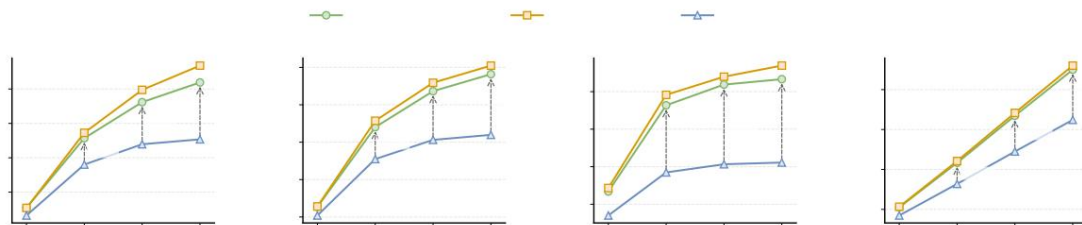


图 4 | 提议长度和延迟开销的影响。在各种块大小下, DSpark 的性能始终优于 DFlash (左侧三幅图)。最右侧的图表明,顺序磁头在服务期间引入的延迟开销极小。

DSpark 的半自回归设计正是基于此。如图 2 所示,DSpark 继承了深度并行绘图器较高的初始接受率(例如,在数学类问题上初始接受率为 0.93)。同时,其轻量级的顺序执行头缓解了并行生成中常见的接受率快速下降的问题。

通过解决这一权衡,DSpark 在整个选秀阶段保持了较高且稳定的条件接受率。

4.3.2. 少量自回归就能发挥巨大作用

基于 4.3.1 节的见解,我们从两个维度探索 DSpark 的架构设计空间:绘图深度 (Transformer 层数) 和方案长度 (块大小)。除非另有说明,本节中的所有实验均使用 Qwen3-4B 作为目标模型,并遵循 4.1 节中详述的评估协议。

绘图器深度。增加 Transformer 层数自然会提升绘图模型的预测能力。为了单独分析这种影响,我们将块大小固定为 7,并将 DSpark 层数从 1 层增加到 5 层,并与 5 层 DFlash 基线进行比较。图 3 汇总了数学、代码和聊天域中可接受的长度。正如预期,DSpark 的性能随深度单调提升,从 1 层到 2 层提升最为显著。值得注意的是,2 层 DSpark 在所有域中均优于 5 层 DFlash 基线。

这表明,通过轻量级顺序头注入局部自回归可以提供非常有利的精度参数权衡,实现比简单地堆叠更深的并行层更好的序列一致性。

提案长度。接下来,我们将草稿深度固定为 5 层,并将草稿长度 (提案长度加上一个锚点标记)在{4, 8, 12, 16}范围内进行缩放,以评估其在更长草稿块上的性能。对于 DSpark,我们评估了默认的马尔可夫头和 RNN 头。[图 4](#)的前三个面板显示,在所有提案长度下,DSpark 的性能始终优于 DFlash。更重要的是,随着草稿长度的增加,性能差距稳步扩大。由于纯并行生成 (DFlash)存在快速接受衰减的问题 ([图2](#)),其边际效用在长块上会降低。DSpark 缓解了这种衰减,使其相对于 DFlash 的优势不断扩大。例如,当 $n=7$ 时,DSpark 在 math、code 和 chat 上的接受长度分别提高了 16%、15%和 18%;当 $n=15$ 时,这些提升分别扩大到 30%、26% 和 22%。此外, RNN 头部相比马尔可夫头部仅提供微弱的额外性能提升,主要体现在较长的提议区域长度上。考虑到其更高的实现复杂度和较差的部署特性,我们默认使用马尔可夫头部。

延迟开销。我们量化了DSpark中顺序生成循环的开销。

[图 4](#)最右侧的面板报告了每轮引擎延迟,该延迟包括一次目标验证、并行草稿块前向传递和串行采样循环,测量时批处理大小为 128。为避免序列长度偏差,所报告的延迟是不同上下文长度 ({512, 1024, 2048, 4096} 个标记)的算术平均值。由于在此批处理大小下,目标模型占据了验证计算时间的大部分,因此顺序块的延迟开销可以忽略不计。因此,尽管将草稿长度从 4 扩展到 16 可以使可接受的长度提高高达30%,但与 DFlash 基线相比,整轮延迟仅增加了 0.2% 到 1.3%。

4.3.3. 更智能地验证,而非耗时更长:置信主管的作用

尽管 DSpark 在较长的草案区块中保持着较高的接受率,但验证整个提案仍然效率低下 ([Hu et al., 2026](#); [Huang et al., 2024](#))。由于[第 4.2 节中提到的固有领域差异](#),开放式聊天中的尾部词仍然面临较高的拒绝风险,使得盲验证成为目标计算资源的浪费。为了评估置信度头能否有效地剪除这些不理想的后缀,我们使用 Qwen3-4B 进行了离线阈值扫描。我们在此单独验证估计器,而将硬件感知前缀调度器 ([第 3.2.2 节](#))留待[第5 节](#)中的实际生产环境评估。

诊断:静态阈值扫描。[图 5](#)绘制了不同置信度阈值下每步平均标记数 (柱状图)和总体接受率 (折线图)。随着阈值的增加,接受率稳步上升,因为估计器会过滤掉最终会被拒绝的标记 (带阴影的柱状图)。这表明置信度头可以识别低值后缀标记,并且这种剪枝在聊天工作负载中最为显著,因为聊天工作负载中较高的熵标记分布限制了固定长度验证的效率。在聊天子图中,提高阈值显著减少了被拒绝的标记,接受率从 45.7% 提高到95.7%。相比之下,结构化任务 (数学和代码)的剪枝程度较轻,保留了更多的草稿标记,接受率分别从 76.9% 提高到 92.5% 和从 67.6% 提高到 92.0%。

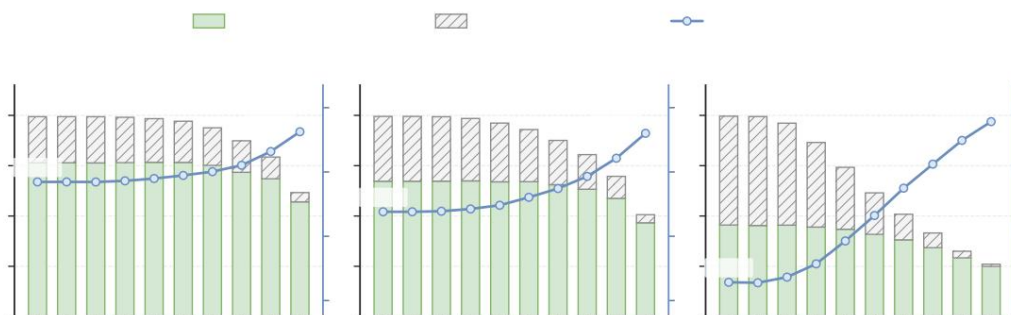


图 5 |置信度阈值扫描。阈值为 0 对应于标准的固定长度验证。随着阈值的增加,整体接受率稳步上升,因为置信度头有效地剔除了最终会被拒绝的令牌(哈希条)。

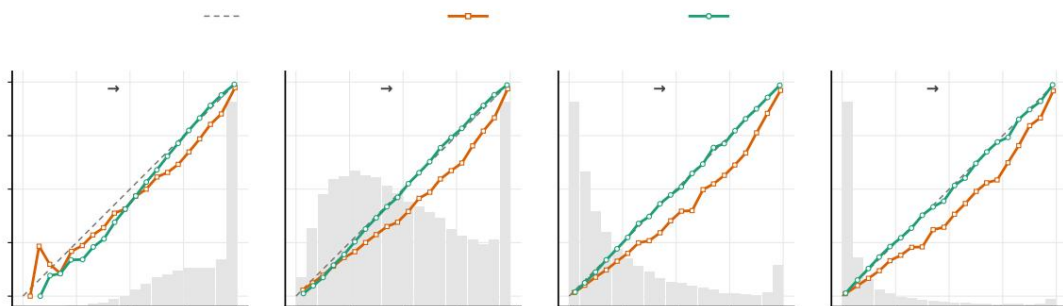


图 6 | Alpaca 数据集的可靠性图。虽然原始置信度估计器具有很强的区分度,但其预测结果本身存在过度自信的问题。应用事后校准有助于将前缀生存概率与经验接受率相匹配。阴影背景直方图表示不同置信区间内样本计数的频率分布。

从静态阈值到校准调度。静态阈值虽然对诊断有用,但在动态服务环境中并非最优,因为它忽略了系统负载:在低并发情况下,验证低置信度令牌的机会成本极低,但在高并发情况下,则会浪费关键的批处理容量。这种负载依赖性促使我们开发硬件感知前缀调度器。如第 3.2 节所述,最大化系统级吞吐量要求置信度模型既要具备强大的预测区分能力,又要具备精确的校准能力,才能准确估计累积生存概率。可靠性图(图6)表明,虽然原始模型实现了强大的区分能力(ROC-AUC (Hanley 和 McNeil, 1982)介于 0.81 到 0.90 之间),但其置信度过高(ECE 为 3%–8%)。应用事后 STS (第 3.2.1 节)可以缓解这种过高置信度,将平均 ECE 降低至约 1%,从而获得可靠的生存概率估计。

5. DSpark 的实际部署

第 4 节阐述了 DSpark 在离线基准测试中的算法优势,而将其与 DeepSeek-V4 (DeepSeek-AI, 2026)等大规模模型结合使用,则会带来额外的优势。

训练和推理都面临着系统层面的挑战。在本节中,我们将介绍 DSpark 的端到端生产流程。我们将详细介绍可扩展的训练机制、部署硬件感知前缀调度器所需的系统级优化 (第 3.2.2 节),以及框架在实际用户流量下的端到端性能。

5.1 可扩展和灵活的培养

DSpark 草案模型与 DeepSeek-V4-Flash 和 DeepSeek-V4-Pro 的预览版 (DeepSeek-AI, 2026) 共同部署。并行骨干网包含三个 MoE 层 (Dai 等人, 2024), 并采用 mHC (Xie 等人, 2026) 和 128 的滑动窗口注意力机制。我们将最大块大小配置为 5, 并使用马尔可夫头进行序列建模。此外,置信度头与草案模型进行端到端训练,随后通过 STS 进行校准,以提供可靠的调度信号。

训练草稿模型需要目标模型的输出分布作为监督。

在完整的文档上下文中评估这两个模型会产生大量的内存占用和工作线程间通信开销。为了解决这些瓶颈问题,我们在内部训练框架 (HAI-LLM) 中实现了两个系统级优化:

- 隐藏状态通信。将目标模型的全部词汇表 logits ($\approx 10^5$) 传输到并行工作节点会造成严重的带宽瓶颈。因此,我们暂时缓存目标模型的前向传播激活值,并且仅传输紧邻语言建模 (LM) 头部之前的隐藏状态。然后,LM 头部投影仅在草稿模型的工作节点上针对采样到的目标位置本地执行。这降低了每个标记的通信复杂度 ($\mathcal{O}(V)$), 其中 V 是隐藏维度。基于锚点的序列打包。为了将草稿模型的计算成本与目标模型的上下文长度解耦,我们从训练序列中采样固定数量的草稿锚点,并将这些独立的预测块打包成密集的训练批次。

我们通过标记级注意力指数而不是标准 2D 掩码来管理这种打包。

这样可以跨多个独立序列和锚点保持精确的因果掩码,避免与标准填充相关的计算和内存开销。

5.2. 硬件感知前缀调度器的实际应用

在第 3.2.2 节中,算法 1 提供了一种理论上可靠且无损的调度机制。

然而,直接将该算法部署到生产环境中会暴露出与现实世界基础设施的两个根本性冲突。首先,该算法假设容量曲线平滑且呈单峰分布,而实际硬件容量 SPS (Steps Per Second) 本质上是离散的,呈现出锯齿状的阶跃式下降 (Yan et al., 2020)。其次,该算法需要对每个步骤的动态草稿令牌进行调度,这与连续 CUDA 图重放 (Fireworks AI, 2023) 和零开销调度 (ZOS) (Zheng et al., 2024; Zhu et al., 2025) 相冲突。

为了权衡系统兼容性、吞吐量和算法正确性,我们对调度器进行了异步操作。由于 ZOS 要求在当前步骤完成之前就知道下一步的批处理大小,同步调度必然会导致 GPU 流水线停滞。因此,我们使用前两步的置信度头输出来估算即将到来的验证容量。在机制上,当前步骤中的候选令牌仍然严格按照其最新的累积置信度分数排序;前两步的历史预测仅用于确定动态截断。

4<https://www.high-flyer.cn/en/blog/hai-llm/>

长度（即批次容量限制）。这实际上将准入过程转化为动态的优先选择。虽然近似容量会引入轻微的时间偏差，但选择机制本质上是保持排名的：最可靠的草稿令牌始终优先进行验证。这种调整完全隐藏了调度延迟，并确保了与 ZOS 的无缝集成。

基于此异步流水线，我们解决了硬件利用率瓶颈问题。为了防止调度器因SPS陡峭的断崖而陷入局部最优解，我们移除了提前停止机制，从而实现了不受约束的全局搜索。通常情况下，这种回顾性搜索会泄露未来的令牌信息，并违反无损保证（附录A）。

然而，我们基于 ZOS 的自适应机制自然而然地避免了这种情况。由于无约束搜索仅评估前两步的历史预测，因此准入决策与当前令牌的实现完全隔离。截断长度本质上仅取决于前两步可用的信息。因此，异步设计形成了一个因果屏障，在保持精确目标分布的同时，最大限度地提高了跨越硬件瓶颈的物理吞吐量。

5.3 高吞吐量和低延迟推理

在解码过程中，生产服务系统必须同时优化两个相互冲突的目标：单次请求延迟和总吞吐量（Kwon et al., 2023; Zhao et al., 2025a; Zhong et al., 2024）。前者决定了单个用户的服务质量，这在基于代理的工作负载中日益重要（Tiwari et al., 2026），而后者决定了并发服务的用户总数。由于推测性解码不可避免地会造成验证计算的浪费，因此它本质上是在权衡这种取舍，以额外的系统计算换取更快的单次请求生成速度。

然而，在我们的部署环境中，每个步骤处理的请求数量经常受到资源限制（例如，每个请求的键值缓存容量固定）和可用用户流量池（例如，强化学习的长尾负载）的制约。因此，有效批处理大小始终远低于GPU的计算饱和阈值。在这种情况下，传统的权衡关系简化了：在并发限制固定的情况下，最大化每个GPU的总令牌吞吐量和最大化每个用户的生成速度（tok/s/user）成为高度相关的目标，而不是相互竞争的目标。

为了实现最大吞吐量，异步调度器（5.2节）会主动将空闲计算资源路由到最有希望的草稿标记。然而，这种动态路由在物理执行层带来了一个严峻的挑战：推理框架必须高效地支持单个批次内的可变长度查询。标准解码内核针对固定查询长度进行了高度优化；简单地处理可变长度的已验证前缀会导致严重的GPU利用率不足，这是由于填充和工作负载分布不均造成的。我们通过将物理执行与逻辑序列跟踪解耦来解决这个问题。在我们的计算内核中，所有不同请求中的标记都被扁平化，并作为独立元素进行相同的处理。然后，复杂的序列内依赖关系通过集成到我们稀疏注意力实现中的标记张量来严格传递。具体来说，在DeepSeek-V4架构上，只有索引注意力内核和压缩内核需要修改以支持这种可变长度路由，从而使动态调度器能够无缝运行，而不会引入底层执行开销。

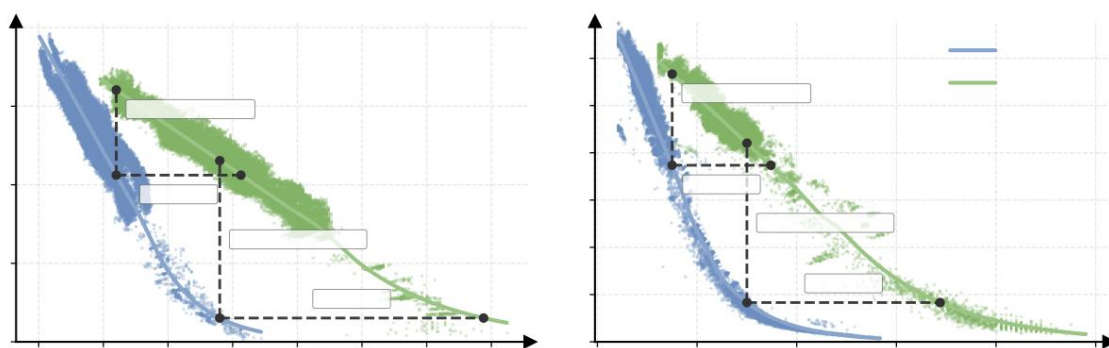


图 7 |吞吐量与 TPS 的关系。在实际流量下,聚合输出令牌吞吐量与每次请求生成速度 (tok/s/user)的关系。在我们的生产部署中,在所测量的流量和引擎配置下,DSpark相对于 MTP-1 基线改进了观察到的吞吐量-交互性平衡。

5.4. 实时用户流量下的性能

我们在 DeepSeek-V4-Flash (预览版)和 DeepSeek-V4-Pro (预览版)的生产服务引擎中,将 DSpark-5 (配置最大草稿长度为 5)与 MTP-1 (DeepSeek-AI,2024)基准进行比较。MTP-1 代表之前的生产环境配置,在 DeepSeek-V4 预览版发布两周后被 DSpark 取代。这种单令牌配置之所以在生产环境中长期保留,是因为部署静态多令牌草稿器 (例如 MTP-3/5)会因过多的验证开销而严重降低高并发情况下的总吞吐量。因此,将 DSpark 与这一既定基准进行比较,可以直接证明其能够在动态服务环境中安全地释放更大草稿块的性能潜力。在所有图中,散点代表直接从实时用户流量中采样的原始遥测数据,捕捉了复杂的真实请求分布,而实线代表拟合的性能边界。

服务帕累托前沿。图 7 展示了系统总吞吐量和单用户生成速度 (交互性)之间的权衡。为了量化 DSpark 在实际部署约束下的性能,我们针对多个交互性 SLA 基准对系统进行了评估。

在这里,服务级别协议 (SLA) 规定了系统必须保证的最低每用户生成速度 (以每秒令牌数计)。

对于 V4-Flash 引擎,我们在 SLA 锚点为 80 和 120 tok/s/user 的情况下评估系统。在适中的 80 tok/s/用户 SLA 下,DSpark 的总吞吐量比 MTP-1 基准提高了 51%。更严格的 120 tok/s/用户 SLA 则代表了一种截然不同的情况:在此约束下,单令牌 MTP-1 基准接近其运行边界,只能维持非常小的并发批次。因此,此时的相对吞吐量比值非常大,DSpark 的总吞吐量名义上提高了 661%。因此,我们将这一高 SLA 点主要解释为 DSpark 扩展了可行交互边界的证据,而不是相对于一个充分利用的基准的乘法加速。在匹配的实际吞吐量水平下 (这提供了更稳定的比较),DSpark 将每个用户的生成速度提高了 60% 到 85%。

V4-Pro 的部署也呈现出相同的模式。在适中的 35 tok/s/用户 SLA 下,DSpark 将总吞吐量提高了 52%。在更严格的 50 tok/s/用户 SLA 下,MTP-1再次进入低并发状态,DSpark 的相对吞吐量优势高达 406%。与 V4-Flash 类似,我们认为这一点表明,DSpark 能够在基线方案无法有效支持的交互目标下维持有效的吞吐量。在系统容量相同的情况下,DSpark 的单用户生成速度提高了 57% 到 78%。总体而言,这些结果表明,DSpark 将观察到的吞吐量-交互性边界向外扩展:它在适中的 SLA 条件下提高了吞吐量,更重要的是,在严格的交互性约束下保持了非退化的服务容量。

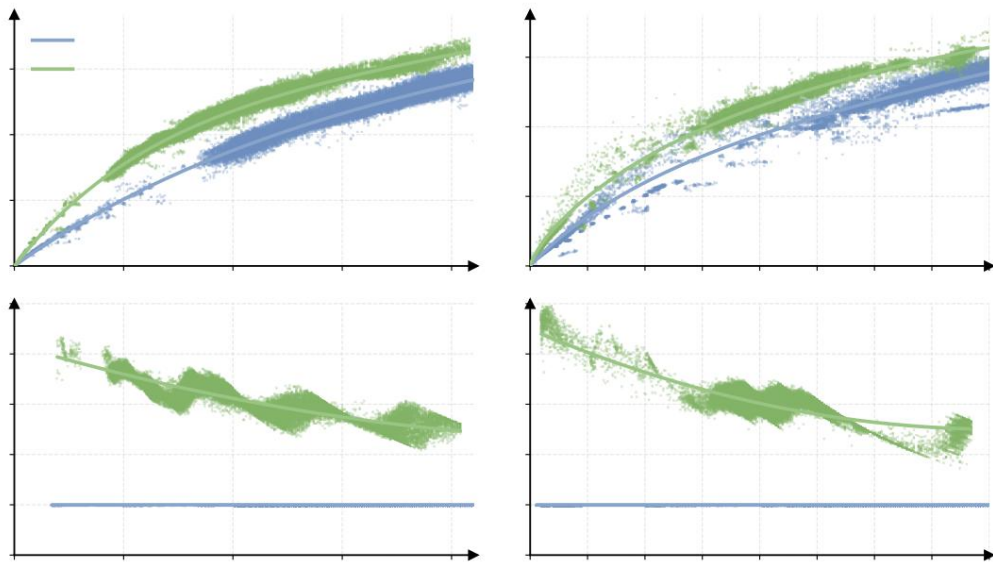


图 8 |负载自适应吞吐量和验证预算。上排 (a,b) :不同系统并发级别下的聚合输出吞吐量。下排 (c,d) :每个请求分配的平均目标验证预算。随着并发负载的增加,动态调度器会自动限制每个请求的验证时长,以防止资源争用。

负载下的吞吐量动态变化。图 8 通过绘制总吞吐量 (上排)和动态验证预算 (下排)与系统并发性的关系图,分析了驱动这些增益的底层机制。

在我们生产部署中常见的适中并发场景下 (V4-Flash 并发请求少于 200 个,V4-Pro 少于 150 个),硬件感知调度器通过分配更长的验证预算来充分利用可用的目标计算能力,将每个请求的验证预算从 MTP-1 的静态 2 个令牌扩展到大约 4-6 个令牌。这种扩展的验证方式使得每次前向传递都能获得更多被接受的令牌,从而直接提升了在帕累托前沿观察到的吞吐量。

随着系统并发量的增加和目标容量的饱和,调度器会动态地限制此预算。平均验证长度会随着负载的增加而平滑地减少,从而确保低置信度的草稿令牌在消耗关键批处理容量之前被剪枝。这种负载感知行为稳定了生产部署:DSpark 最大化

在流量较小时,可有效利用空闲计算能力;而在流量较大时,可有效保持关键批处理能力。

局限性。虽然前缀调度器最大限度地减少了目标模型验证的浪费,但 DSpark 仍然会产生固定的草案端成本,用于通过并行骨干网生成初始令牌区块。

对于接受率本身就较低的复杂查询,这种预先进行的草稿计算是无法恢复的。未来的优化可以在草稿模型中引入难度感知提前退出机制,使这类请求能够绕过完整的块生成过程。

6. 相关工作

推测性解码算法。推测性解码通过将词元提议与验证解耦来加速自回归生成。基于早期的分块方法 (Ge et al., 2022; Stern et al., 2018; Sun et al., 2021; Xia et al., 2023), 现代方法采用拒绝采样来精确地保留目标模型的分布 (Chen et al., 2023; Leviathan et al., 2023)。由于推理速度的提升直接取决于生成器的效率和准确性,因此大量的研究都集中在优化其架构上。除了使用独立的小型语言模型 (Chen 等人, 2023; Leviathan 等人, 2023) 之外,后续工作将多标记词头或特征外推器直接集成到目标模型中 (Ankner 等人, 2024; Cai 等人, 2024, 2025; DeepSeek-AI, 2024; Gloeckle 等人, 2024; Li 等人, 2024b,c, 2026b; Zhang 等人, 2025)。其他策略包括通过提前退出进行自我推测 (Elhoushi et al., 2024; Liu et al., 2024a; Xia et al., 2025; Zhang et al., 2024)、动态词汇压缩 (Williams et al., 2026; Zhao et al., 2025b)、提示查找 (Saxena, 2023; Somasundaram et al., 2025)、后缀自动机 (Hu et al., 2025) 和检索 (He et al., 2023; Shen et al., 2026)。为了消除草稿生成过程中的顺序瓶颈,最近的方法提出了并行或分块生成。P-EAGLE 并行化了 EAGLE 式的草稿生成过程 (Hui 等人, 2026), 而 PARD、DART 和 DFlash 则利用扩散启发式预测,在单次前向传播中生成整个块 (An 等人, 2026; Chen 等人, 2026; Liu 等人, 2026a), DDTree 随后将其扩展为可验证的草稿树 (Ringel 和 Romano, 2026)。并行研究也改进了 DFlash: Domino (Huang 等人, 2026a) 引入了一个概念上类似于我们 RNN Head 的因果编码器,而 DFlare (Zhang 等人, 2026a) 则通过逐层融合解决了条件瓶颈问题。

面向推测性解码的系统感知调度。除了选题架构之外,另一项研究工作侧重于确定每一轮中生成或验证的推测性令牌的最佳数量。为此,各种方法利用置信度启发式 (Du et al., 2024; Li et al., 2024b; Liu et al., 2026c; Mamou et al., 2024; Wen and Feng, 2026)、学习型接受预测器 (Huang et al., 2024; Zacks917, 2026) 或多臂老虎机策略 (Liu et al., 2026b) 动态调整选题长度。此外,鉴于推测解码本质上是一个系统级调度问题,最近的研究通过根据实时系统负载和请求优先级调整推测预算来优化整体吞吐量和延迟 (AngelSlim Team, 2026; Hu 等人, 2026; Huang 等人, 2026b; Li 等人, 2026a; Liu 等人, 2024c; Miao 等人, 2024; Sadhukhan 等人, 2025; Wu 等人, 2025)。

并行生成。并行生成标记的模型提供的解码延迟几乎与输出长度无关,使其成为自回归解码的一种有吸引力的替代方案。

非自回归变换器 (NAT, Gu 等人, 2018) 开创了这一方向,它通过一次迭代独立预测所有位置。然而,这迫使模型对所有位置进行平均。

合理的模式,通常会产生混合了来自不同有效序列的片段的输出。

为了解决这一局限性,目前出现了两种主要的研究方向。一种方向保留了单遍架构,但改变了模型所看到的内容或训练方式:引入潜在变量作为条件输入,引导所有位置趋向一致的输出 (Gu et al., 2018; Kaiser et al., 2018; Ma et al., 2019),或者放宽训练目标,使模型专注于产生单一的连贯输出,而不是对所有有效备选方案的完整分布进行建模 (Du et al., 2021; Qian et al., 2021; Shao et al., 2021, 2023)。另一种方法是通过迭代重预测 (Austin 等人, 2021a; Ghazvininejad 等人, 2019; Li 等人, 2022)、块级自回归 (Arriola 等人, 2025; Wang 等人, 2018)或结构化输出层 (如 CRF (Sun 等人, 2019)、CTC (Libovický 和 Helcl, 2018; Saharia 等人, 2020)、HMM (Huang 等人, 2022b)和 PCFG (Gui 等人, 2023))重新引入有限的序列依赖性。

推测性解码进一步要求设计者必须为拒绝采样规则提供精确的逐词概率。由于迭代细化、潜在边缘化或全局归一化,上述大多数技术无法轻易提供此类概率。例如,在与我们设计密切相关的 CRF-NAT (Sun 等人, 2019)中,也在并行隐藏状态上放置了一个顺序模块,但其全局归一化的配分函数阻碍了精确的逐词概率计算。类似地,当将 CTC 输出层适配到并行推测性解码时,由于对齐路径的潜在边缘化,CTC-draafter (Wen 等人, 2024)只能进行贪婪验证。DSpark 通过将顺序校正保持局部化来规避这些限制,因此逐词概率仍然是精确的 softmax 评估值。

7. 结论

本文提出了一种名为 DSpark 的推测性解码框架,旨在克服高并发生产环境中大型语言模型推理的结构和系统级瓶颈。在算法层面,DSpark 引入了一种半自回归生成范式,将计算密集型并行骨干与轻量级顺序头相结合,以缓解独立并行生成器后缀快速衰减的问题。在系统层面,我们将验证长度选择建模为一个全局吞吐量最大化问题,并采用了一种硬件感知的前缀调度器,该调度器能够根据校准后的生存概率和实时引擎负载动态调整目标模型的验证预算。大量的离线评估表明,DSpark 在多个领域均显著优于最先进的自回归和并行基线方法。此外,它在 DeepSeek-V4 中的实际部署验证了其在生产服务中的实用价值:通过智能地管理验证开销,DSpark 在重负载下保持了强大的并发性,持续加快了每个用户的生成速度,并有效地将 LLM 服务的帕累托前沿向外移动。

参考

T. Abramovich, M. Ashkenazi, I. Putterman, B. Chislett, T. Mitra, BD Rouhani, R. Zilberstein 和 Y. Geifman. “Speed-bench: 一个统一且多样化的推测解码基准测试”。arXiv预印本 arXiv:2604.09557, 2026 年。

Z. An, H. Bai, Z. Liu, D. Li 和 E. Barsoum. “PARD: 利用低成本并行草稿模型自适应加速 LLM 推理”。第十四届国际学习表征会议, 2026 年。网址: <https://openreview.net/forum?id=XbOyv7iVGL>。

AngelSlim 团队.D-Cut:用于推测性解码的自适应验证深度剪枝,2026 年。
URL <https://angelslim.readthedocs.io/zh-cn/latest/dcut.html>。

Z. Ankner,R. Parthasarathy,A. Nrusimha,C. Rinard,J. Ragan-Kelley 和 W. Brandon。“Hydra:用于美杜莎解码的顺序依赖草稿头”。载于2024 年第一届语言建模会议。网址: <https://openreview.net/forum?id=FbhjirzyJG>。

M. Arriola,SS Sahoo,A. Gokaslan,Z. Yang,Z. Qi,J. Han,JT Chiu 和 V. Kuleshov。
块扩散:在自回归和扩散语言模型之间进行插值。第十三届国际学习表征会议, 2025 年。网址: <https://openreview.net/forum?id=tyEyYT267x>。

J. Austin,DD Johnson,J. Ho,D. Tarlow 和 R. van den Berg。“离散状态空间中的结构化去噪扩散模型”。载于 A. Beygelzimer、Y. Dauphin,P. Liang 和 JW Vaughan 编辑的《神经信息处理系统进展》, 2021a。网址:<https://openreview.net/forum?id=h7-XixPCAL>。

J. Austin,A. Odena,M. Nye,M. Bosma,H. Michalewski,D. Dohan,E. Jiang,C. Cai,M. Terry、Q. Le 等。使用大型语言模型进行程序综合。arXiv 预印本 arXiv:2108.07732, 2021b。

T. Cai,Y. Li,Z. Geng,H. Peng,JD Lee,D. Chen 和 T. Dao。“Medusa:具有多个解码头的简单 LLM 推理加速框架”。载于 R. Salakhutdinov,Z. Kolter、K. Heller,A. Weller,N. Oliver,J. Scarlett 和 F. Berkenkamp 编辑的《第 41 届国际机器学习会议论文集》,《机器学习研究论文集》第 235 卷,第 5209–5235 页。PMLR, 2024 年 7 月 21–27 日。网址:<https://proceedings.mlr.press/v235/cai24b.html>。

蔡Y.,梁X.,王X.,马J.,H.梁,J.罗,X.左,L.段,Y.尹,X.陈。
Fastmtp:通过增强的多标记预测加速 llm 推理,2025 年。URL <https://arxiv.org/abs/2509.18362>。

C. Chen,S. Borgeaud,G. Irving,J.-B. Lespiau,L. Sifre 和 J. Jumper。“利用推测采样加速大型语言模型解码”。arXiv 预印本 arXiv:2302.01318,2023 年。

J. Chen,Y. Liang 和 Z. Liu. Dflash:用于闪存推测解码的块扩散。arXiv 预印本 arXiv:2602.06036,2026 年。

M. Chen,J. Tworek,H. Jun,Q. Yuan,HP de Oliveira Pinto,J. Kaplan,H. Edwards,Y. Burda、N. Joseph,G. Brockman,A. Ray,R. Puri,G. Krueger,M. Petrov,H. Khlaaf,G. Sastry,P. Mishkin、B. Chan,S. Gray,N. Ryder,M. Pavlov,A. Power,L. Kaiser,M. Bavarian,C. Winter,P. Tillet,FP Such,D. Cummings,M. Plappert,F. Chantzis,E. Barnes,A. Herbert-Voss,WH Guss、A. Nichol,A. Paino,N. Tezak,J. Tang,I. Babuschkin,S. Balaji,S. Jain,W. Saunders,C. Hesse、AN Carr,J. Leike、J. Achiam,V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever 和 W. Zaremba。评估在代码上训练的大型语言模型,2021 年。

Y. Cheng,A. Zhang,X. Zhang,C. Wang 和 Y. Wang。用于大型语言模型中快速推测解码的循环草稿器,2024。URL <https://arxiv.org/abs/2403.09919>。

K. Cobbe,V. Kosaraju,M. Bavarian,M. Chen,H. Jun,L. Kaiser,M. Plappert,J. Tworek,J. Hilton、R. Nakano,C. Hesse 和 J. Schulman。“训练验证器解决数学应用题”。arXiv预印本 arXiv:2110.14168,2021 年。

D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu 等。Deepseekmoe: 迈向混合专家语言模型中的终极专家专精。载于第 62 届计算语言学协会年会论文集 (第 1 卷: 长论文), 第 1280–1297 页, 2024 年。

DeepSeek-AI。Deepseek-v3 技术报告。arXiv 预印本 arXiv:2412.19437, 2024。

DeepSeek-AI。Deepseek-v4: 迈向高效的百万级上下文智能, 2026 年。
URL <https://arxiv.org/abs/2606.19348>。

C. Du, Z. Tu 和 J. Jiang。非自回归机器翻译的与顺序无关的交叉熵。载于 M. Meila 和 T. Zhang 编辑的《第 38 届国际机器学习会议论文集》, 《机器学习研究论文集》第 139 卷, 第 2849–2859 页。PMLR, 2021 年 7 月 18–24 日。网址: <https://proceedings.mlr.press/v139/du21c.html>。

C. Du, J. Jiang, X. Yuanchen, J. Wu, S. Yu, Y. Li, S. Li, K. Xu, L. Nie, Z. Tu 和 Y. You。GliDe with a CaPE: A low-hassle method to accelerate speculative decode. In R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett 和 F. Berkenkamp (editors), Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 11704–11720. PMLR, 2024 年 7 月 21–27 日。URL: <https://proceedings.mlr.press/v235/du24c.html>。

M. Elhoushi, A. Shrivastava, D. Liskovich, B. Hosmer, B. Wasti, L. Lai, A. Mahmoud, B. Acun, S. Agarwal, A. Roman, A. Aly, B. Chen 和 C.-J. Wu。“Layerskip: 实现早期退出推理和自推测解码”。载于第 62 届计算语言学协会年会论文集 (第 1 卷: 长论文), 第 12622–12642 页。计算语言学协会, 2024 年。doi:10.18653/v1/2024.acl-long.681。URL: <http://dx.doi.org/10.18653/v1/2024.acl-long.681>。

Fireworks AI。速度与 Python: 二选一。CUDA 图如何助力 Python 代码快速迭代, 实现深度学习。<https://fireworks.ai/blog/speed-python-pick-two-how-cuda-graphs-enable-fast-python-code-for-deep-learning> 2023 年 8 月。访问日期: 2026 年 6 月 22 日。

T. Ge, H. Xia, X. Sun, S.-Q. Chen 和 F. Wei。采用激进解码的 seq2seq 生成的无损加速。arXiv 预印本 arXiv:2205.10350, 2022。

M. Ghazvininejad, O. Levy, Y. Liu 和 L. Zettlemoyer。“Mask-predict: 条件掩码语言模型的并行解码”。载于 K. Inui, J. Jiang, V. Ng 和 X. Wan 编辑的《2019 年自然语言处理实证方法会议暨第九届自然语言处理国际联合会会议论文集 (EMNLP-IJCNLP)》, 第 6112–6121 页, 中国香港, 2019 年 11 月。计算语言学协会。doi: 10.18653/v1/D19-1633。网址: <https://aclanthology.org/D19-1633/>。

F. Gloeckle, B. Youbi Idrissi, B. Roziere, D. Lopez-Paz 和 G. Synnaeve。“通过多标记预测实现更好更快的大规模语言模型”。载于 R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett 和 F. Berkenkamp 编辑的《第 41 届国际机器学习会议论文集》, 《机器学习研究论文集》第 235 卷, 第 15706–15734 页。PMLR, 2024 年 7 月 21–27 日。网址: <https://proceedings.mlr.press/v235/gloeckle24a.html>。

Google DeepMind。Gemma 4 模型卡。https://ai.google.dev/gemma/docs/core/model_card_4 2026 年。访问日期: 2026 年 6 月 11 日。

J. Gu, J. Bradbury, C. Xiong, VO Li 和 R. Socher. “非自回归神经机器翻译”。在2018 年国际学习表征会议上。网址: <https://openreview.net/forum?id=B1l8BtlCb>。

S. Gui, C. Shao, Z. Ma, X. Zhang, Y. Chen 和 Y. Feng. “基于概率上下文无关文法的非自回归机器翻译”。载于2023 年第三十七届神经信息处理系统会议。网址: <https://openreview.net/forum?id=LloZFVwWvj>。

C. Guo, G. Pleiss, Y. Sun 和 KQ Weinberger. 关于现代神经网络的校准。国际机器学习会议, 第 1321-1330 页。PMLR, 2017 年。

JA Hanley 和 BJ McNeil. 受试者工作特征 (ROC) 曲线下面积的意义和用途。放射学, 143(1):29-36, 1982。doi: 10.1148/radiology.143.1.7063747。

Z. He, Z. Zhong, T. Cai, JD Lee 和 D. He. Rest: 基于检索的推测解码, 2023。

X. Hu, Y. Shen, B. Zhang, H. Zhang, J. Dai, S. Ge, L. Chen, Y. Li 和 M. Wan. Echo: 面向高并发场景的稀疏门控弹性推测解码。arXiv预印本 arXiv:2604.09603, 2026。

Y. Hu, K. Wang, X. Zhang, F. Zhang, C. Li, H. Chen 和 J. Zhang. SAM 解码: 基于后缀自动机的推测解码。载于 W. Che, J. Nabende, E. Shutova 和 MT Pilehvar 编辑的《第 63 届计算语言学协会年会论文集 (第 1 卷: 长论文)》, 第 12187-12204 页, 奥地利维也纳, 2025 年 7 月。计算语言学协会。ISBN 979-8-89176-251-0。doi:10.18653/v1/2025.acl-long.595。

URL <https://aclanthology.org/2025.acl-long.595/>。

F. Huang, T. Tao, H. Zhou, L. Li 和 M. Huang. 关于非自回归Transformer的学习。载于 K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu 和 S. Sabato 编辑的《第39届国际机器学习会议论文集》, 《机器学习研究论文集》第162卷, 第9356-9376页。PMLR, 2022年7月17-23日。网址: <https://proceedings.mlr.press/v162/huang22k.html>。

F. Huang, H. Zhou, Y. Liu, H. Li 和 M. Huang. “用于非自回归机器翻译的有向无循环Transformer”。载于 K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu 和 S. Sabato 编辑的《第39届国际机器学习会议论文集》, 《机器学习研究论文集》第162卷, 第9410-9428页。PMLR, 2022年7月17-23日。网址: <https://proceedings.mlr.press/v162/huang22m.html>。

J. Huang, Y. Zhang, Q. Zhang, H. Lin, H. Xu 和 L. Zhang. 《Domino: 在推测性解码中将因果建模与自回归草案解耦》, 2026a。网址: <https://arxiv.org/abs/2605.29707>。

K. Huang, X. Guo 和 M. Wang. Specdec++: 通过自适应增强推测解码候选长度。arXiv 预印本 arXiv:2405.19715, 2024。

K. Huang, H. Wu, Z. Shi, H. Zou, M. Yu 和 Q. Shi. Adaspec: 面向快速、慢感知的大型语言模型服务的自适应推测解码。载于 2025 年 ACM 云计算研讨会论文集 (SoCC '25), 第 361-374 页, 美国纽约州纽约市, 2026b。美国计算机协会 (ACM)。ISBN 9798400722769。doi:10.1145/3772052.3772239。URL: <https://doi.org/10.1145/3772052.3772239>。

M. Hui, X. Huang, JC Salas, Y. Sun, N. Pemberton, X. Song, A. Khetan 和 G. Karypis. P-eagle: 具有可扩展训练的平行起草鹰, 2026 年。URL <https://arxiv.org/abs/2602.01469>。

D. Israel, G. Van den Broeck 和 A. Grover. “通过自适应并行解码加速扩散 LLMs”。神经信息处理系统进展, 38:52870–52888, 2026。

N. Jain, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen 和 I. Stoica. Livecodebench: 对大型语言模型进行全面、无污染的代码评估。在国际学习表征会议论文集, 第 2025 卷, 第 58791-58831 页, 2025 年。

L. Kaiser, S. Bengio, A. Roy, A. Vaswani, N. Parmar, J. Uszkoreit 和 N. Shazeer. “使用离散潜在变量的序列模型快速解码”。载于 J. Dy 和 A. Krause 编辑的《第 35 届国际机器学习会议论文集》, 《机器学习研究论文集》第 80 卷, 第 2390–2399 页。PMLR, 2018 年 7 月 10–15 日。网址: <https://proceedings.mlr.press/v80/kaiser18a.html>。

W. Kwon, Z. Li, S. Zhuang, Y. Shen, L. Cheng, CH Yu, J. Gonzalez, H. Zhang 和 I. Stoica. 针对具有页面注意力机制的大型语言模型服务的高效内存管理。第 29 届操作系统原理研讨会论文集, 第 611-626 页, 2023 年。

Y. Leviathan, M. Kalman 和 Y. Matias. “基于推测解码的 Transformer 快速推理”。载于 A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato 和 J. Scarlett 编辑的《第 40 届国际机器学习会议论文集》, 《机器学习研究论文集》第 202 卷, 第 19274–19286 页。PMLR, 2023 年 7 月 23–29 日。网址: <https://proceedings.mlr.press/v202/leviathan23a.html>。

G. Li, Z. Fu, M. Fang, Q. Zhao, M. Tang, C. Yuan 和 J. Wang. “Diffuspec: 解锁用于推测性解码的扩散语言模型”, 2025a。网址: <https://arxiv.org/abs/2510.02358>。

R. Li, Z. Zhang, L. Zhang, H. Wang, X. Fu 和 Z. Lai. Nightjar: 面向大型语言模型服务的动态自适应推测解码, 2026a。URL <https://arxiv.org/abs/2512.22420>。

T. Li, W.-L. Chiang, E. Frick, L. Dunlap, B. Zhu, J. E. Gonzalez 和 I. Stoica. “从实时数据到高质量基准测试: arena-hard 管道, 2024 年 4 月”。URL <https://lmsys.org/blog/2024-04-19-arena-hard/>。

T. Li, W.-L. Chiang, E. Frick, L. Dunlap, T. Wu, B. Zhu, J. E. Gonzalez 和 I. Stoica. 从众包数据到高质量基准测试: Arena-hard 和基准测试构建器管道。A. Singh, M. Fazel, D. Hsu, S. Lacoste-Julien, F. Berkenkamp, T. Maharaj, K. Wagstaff 和 J. Zhu 编辑, 《第 42 届国际机器学习会议论文集》, 《机器学习研究论文集》第 267 卷, 第 34209–34231 页。PMLR, 2025 年 7 月 13–19 日。URL: <https://proceedings.mlr.press/v267/li25h.html>。

XL Li, J. Thickstun, I. Gulrajani, P. Liang 和 T. Hashimoto. “扩散语言模型改进可控文本生成”。载于 A. Agarwal, D. Belgrave 和 K. Cho 编辑的《神经信息处理系统进展》, 2022 年。网址: <https://openreview.net/forum?id=3s9lrEsjLyk>。

Y. Li, F. Wei, C. Zhang 和 H. Zhang. “EAGLE-2: 利用动态草稿树更快地推断语言模型”。载于 Y. Al-Onaizan, M. Bansal 和 Y.-N. Chen 编辑的《2024 年自然语言处理实证方法会议论文集》, 第 7421–7432 页, 美国佛罗里达州迈阿密, 2024 年 11 月。计算语言学协会。doi:10.18653/v1/2024.emnlp-main.422。网址: <https://aclanthology.org/2024.emnlp-main.422/>。

Y. Li, F. Wei, C. Zhang 和 H. Zhang. “EAGLE: 推测性采样需要重新思考特征不确定性”。载于 R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett 和 F. Berkenkamp 编辑的《第 41 届国际机器学习会议论文集》,《机器学习研究论文集》第 235 卷,第 28935–28948 页。

PMLR, 2024 年 7 月 21–27 日。网址: <https://proceedings.mlr.press/v235/li24bt.html>。

Y. Li, F. Wei, C. Zhang 和 H. Zhang. “EAGLE-3: 通过训练时测试扩展大型语言模型的推理加速”。载于《第三十九届神经信息处理系统年会》, 2026b。网址: <https://openreview.net/forum?id=4e xx1hUffq>。

J. Libovický 和 J. Helcl. 基于连接主义时间分类的端到端非自回归神经机器翻译。载于 E. Riloff, D. Chiang, J. Hockenmaier 和 J. Tsujii 编辑的《2018 年自然语言处理实证方法会议论文集》,第 3016–3021 页,比利时布鲁塞尔,2018 年 10 月至 11 月。计算语言学协会。doi: 10.18653/v1/D18-1336。网址: <https://aclanthology.org/D18-1336/>。

H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever 和 K. Cobbe. “让我们一步一步地验证”。第十二届国际学习表征会议, 2024 年。网址: <https://openreview.net/forum?id=v8L0pN6EOi>。

F. Liu, Y. Tang, Z. Liu, Y. Ni, D. Tang, K. Han 和 Y. Wang. “Kangaroo: 一种通过双重提前退出加速 LLMS 的无损自推测解码”。载于 A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak 和 C. Zhang 编辑的《神经信息处理系统进展》,第 37 卷,第 11946–11965 页。Curran Associates, Inc., 2024a。doi: 10.52202/079017-0381。URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/16336d94a5ffca8de019087ab7fe403f-Paper-Conference.pdf。

F. Liu, X. Li, K. Zhao, Y. Gao, Z. Zhou, Z. Zhang, Z. Wang, W. Dou, S. Zhong 和 C. Tian. Dart: 受扩散启发的推测性解码,用于快速 llm 推理,2026a。URL <https://arxiv.org/abs/2601.19278>。

H. Liu, J. Huang, Z. Jia, Y. Park 和 Y.-X. Wang. “非强盗: LLMS 推测解码中可证明无悔的草稿选择”,2026b。URL: <https://arxiv.org/abs/2510.20064>。

T. Liu, Q. Lv, Y. Shen, X. Sun 和 X. Sun. Talon: 具有自适应标记树的置信度感知推测解码。arXiv 预印本 arXiv:2601.07353,2026c。

X. Liu, C. Daniel, L. Hu, W. Kwon, Z. Li, X. Mo, A. Cheung, Z. Deng, I. Stoica 和 H. Zhang. 利用 goodput 优化推测解码以服务大型语言模型。arXiv 预印本, arXiv-2406 页,2024b。

X. Liu, J. Park, L. Hu, W. Kwon, Z. Li, C. Zhang, K. Du, X. Mo, K. You, A. Cheung 等。Turbospec: 用于优化 llm 服务吞吐量的闭环投机控制系统。arXiv 预印本 arXiv:2406.14066,2024c。

X. Ma, C. Zhou, X. Li, G. Neubig 和 E. Hovy. “FlowSeq: 基于生成流的非自回归条件序列生成”。载于 K. Inui, J. Jiang, V. Ng 和 X. Wan 编辑的《2019 年自然语言处理实证方法会议暨第九届自然语言处理国际联合会议论文集》。

(EMNLP-IJCNLP),第 4282-4292 页,中国香港,2019 年 11 月。计算语言学协会。doi:10.18653/v1/D19-1437。网址:
<https://aclanthology.org/D19-1437/>。

J. Mamou,O. Pereg,D. Korat,M. Berchansky,N. Timor,M. Wasserblat 和 R. Schwartz。
动态推测前瞻加速大型语言模型的推测解码。

在 M. Rezagholizadeh,P. Passban,S. Samiee,V. Partovi Nia,Y. Cheng,Y. Deng,Q. Liu 和 B. Chen 编辑的《第四届 NeurIPS 高效自然语言和语音处理研讨会论文集》(机器学习研究论文集第 262 卷)中,第 456-467 页。PMLR,2024 年 12 月 14 日。URL: <https://proceedings.mlr.press/v262/mamou24a.html>。

X. Miao,G. Oliaro,Z. Zhang,X. Cheng,Z. Wang,Z. Zhang,RYY Wong,A. Zhu,L. Yang, X. Shi 等。Specinfer:利用基于树的推测推理和验证加速大型语言模型服务。载于第 29 届 ACM 编程语言和操作系统体系结构支持国际会议论文集,第 3 卷,第 932-949 页,2024 年。

MP Naeini,G. Cooper 和 M. Hauskrecht。使用贝叶斯分箱获得良好校准的概率。载于 AAAI 人工智能会议论文集,第 29 卷,2015 年。

Y. Ovadia,E. Fertig,J. Ren,Z. Nado,D. Sculley,S. Nowozin,J. Dillon,B. Lakshminarayanan 和 J. Snoek。“你能相信你的模型的不确定性吗?评估数据集偏移下的预测不确定性”。神经信息处理系统进展,32,2019。

L. Qian,H. Zhou,Y. Bao,M. Wang,L. Qiu,W. Zhang,Y. Yu 和 L. Li。用于非自回归神经机器翻译的 Glancing Transformer。载于 C. Zong,F. Xia,W. Li 和 R. Navigli 编辑的《第 59 届计算语言学协会年会暨第 11 届自然语言处理国际联合会议论文集(第 1 卷:长篇论文)》,第 1993-2003 页,在线,2021 年 8 月。计算语言学协会。doi:10.18653/v1/2021.acl-long.155。网址: <https://aclanthology.org/2021.acl-long.155/>。

Y. Ren,J. Liu,X. Tan,Z. Zhao,S. Zhao 和 T.-Y. Liu。“序列生成的非自回归模型研究”。载于第 58 届计算语言学协会年会论文集,第 149-159 页,2020 年。

L. Ringel 和 Y. Romano。利用块扩散草稿树加速推测性解码。
[arXiv 预印本 arXiv:2604.12989](https://arxiv.org/abs/2604.12989), 2026。

R. Sadhukhan,J. Chen,Z. Chen,V. Tiwari,R. Lai,J. Shi,IE-H. Yen,A. May,T. Chen 和 B. Chen。
Magicdec:利用推测性解码打破长上下文生成的延迟-吞吐量权衡。第十三届国际学习表征会议,2025 年。网址: <https://openreview.net/forum?id=CS2JWaziYr>。

C. Saharia,W. Chan,S. Saxena 和 M. Norouzi。“基于潜在对齐的非自回归机器翻译”。载于 B. Webber,T. Cohn,Y. He 和 Y. Liu 编辑的《2020 年自然语言处理实证方法会议论文集(EMNLP)》,第 1098-1108 页,2020 年 11 月在线发表。计算语言学协会。doi:10.18653/v1/2020.emnlp-main.83。网址: <https://aclanthology.org/2020.emnlp-main.83/>。

- J. Sandler, J. Christopher, T. Hartvigsen 和 F. Fioretto。"Specdiff-2:扩展扩散绘图器对齐以实现更快的推测性解码"。第九届机器学习与系统会议,2026 年。网址: <https://openreview.net/forum?id=o42VU86ZsV>。
-
- A. Saxena,《提示查找解码》,2023 年 11 月。网址: <https://github.com/apoorvumang/prompt-lookup-decoding/>。
- C. Shao, Y. Feng, J. Zhang, F. Meng 和 J. Zhou. 非自回归神经机器翻译的序列级训练。计算语言学, 47(4):891–925, 2021 年 12 月。doi: 10.1162/coli_a_00421. URL <https://aclanthology.org/2021.cl-4.29/>。
- C. Shao, Z. Ma, M. Zhang 和 Y. Feng. 超越 MLE:用于文本生成的凸学习。在 2023 年第三十七届神经信息处理系统会议上。网址: <https://openreview.net/forum?id=sla7V80uWA>。
- Y. Shen, T. Liu, X. Hu, Q. Kong, B. Zhang, J. Dai, J. Zhang, S. Ge, L. Chen, Y. Li, M. Wan 和 C. Wang。"少写草稿,多检索:用于推测性解码的混合树构建",2026。
URL <https://arxiv.org/abs/2605.20104>。
- S. Somasundaram, A. Phukan 和 A. Saxena。"PLD+:利用语言模型工件加速语言学习模型推理"。载于 L. Chiruzzo, A. Ritter 和 L. Wang 编,《计算语言学协会研究成果:NAACL 2025》,第 6090–6104 页,新墨西哥州阿尔伯克基,2025 年 4 月。计算语言学协会。ISBN 979-8-89176-195-7。doi:10.18653/v1/2025.findings-naacl.338。网址: <https://aclanthology.org/2025.findings-naacl.338/>。
- M. Stern, N. Shazeer 和 J. Uszkoreit。"深度自回归模型的分块并行解码"。载于 S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi 和 R. Garnett 编辑的《神经信息处理系统进展》第 31 卷。Curran Associates, Inc., 2018 年。网址: https://proceedings.neurips.cc/paper_files/paper/2018/file/c4127b9194fe8562c64dc0f5bf2c93bc-Paper.pdf。
- X. Sun, T. Ge, F. Wei 和 H. Wang. 基于浅层激进解码的即时语法错误纠正。载于 C. Zong, F. Xia, W. Li 和 R. Navigli 编辑的《第 59 届计算语言学协会年会暨第 11 届自然语言处理国际联合会议论文集(第 1 卷:长篇论文)》,第 5937–5947 页,2021 年 8 月在线发表。计算语言学协会。doi: 10.18653/v1/2021.acl-long.462。网址: <https://aclanthology.org/2021.acl-long.462/>。
-
- Z. Sun, Z. Li, H. Wang, D. He, Z. Lin 和 Z. Deng。"序列模型的快速结构化解码"。载于 H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox 和 R. Garnett 编辑的《神经信息处理系统进展》第 32 卷。Curran Associates, Inc., 2019 年。网址: https://proceedings.neurips.cc/paper_files/paper/2019/file/74563ba21a90da13dacf2a73e3ddefa7-Paper.pdf。
- R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang 和 T. Hashimoto。斯坦福羊驼:一种遵循指令的羊驼模型。https://github.com/tatsu-lab/stanford_alpaca 2023 年。
- S. Tiwari, T. Chugh, N. Rickert, S. Peter, R. Mahajan 和 H. Shen。"Cachewise:理解工作负载并优化 kvcache 管理以高效地服务于 LLM 编码代理"。arXiv 预印本 arXiv:2606.16824, 2026 年。
-

C. Wang, J. Zhang 和 H. Chen. 半自回归神经机器翻译。载于 E. Riloff、D. Chiang、J. Hockenmaier 和 J. Tsujii 编辑的《2018 年自然语言处理实证方法会议论文集》,第 479-488 页,比利时布鲁塞尔, 2018 年 10 月-11 月。计算语言学协会。doi: 10.18653/v1/D18-1044。网址: <https://aclanthology.org/D18-1044/>。

Z. Wang, D. Ma, X. Huang, D. Cai, T. Lan, J. Xu, H. Mi, X. Tang 和 Y. Wang。 “手动解码的终结:迈向真正的端到端语言模型”。第十四届国际学习表征会议, 2026 年。网址: <https://openreview.net/forum?id=cPTgQDMD5p>。

Z. Wen 和 Y. Feng. Specbound:具有分层置信度的自适应有界自推测校准,2026。URL <https://arxiv.org/abs/2604.12247>。

Z. Wen, S. Gui 和 Y. Feng. 基于 CTC 的草案模型用于 LLM 推理加速的推测性解码。载于 A. Globerson、L. Mackey、D. Belgrave、A. Fan、U. Paquet、J. Tomczak 和 C. Zhang 编辑的《神经信息处理系统进展》第 37 卷,第 92082-92100 页。Curran Associates, Inc., 2024 年。doi:10.52202/079017-2923。URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/a79054a9da91d73ed3cb1a9e87d7cd2d-Paper-Conference.pdf。

M. Williams, YD Kwon, R. Li, A. Kouris 和 SI Venieris。 “使用推测词汇进行推测解码”。arXiv 预印本 arXiv:2602.13836, 2026 年。

Z. Wu, Z. Zhou, A. Verma, A. Prakash, D. Rus 和 BKH Low。 “TETRIS:批量推测解码的最优草稿标记选择”。载于 W. Che, J. Nabende, E. Shutova 和 MT Pile-hvar 编辑的《第 63 届计算语言学协会年会论文集 (第 1 卷:长篇论文)》,第 33329-33345 页,奥地利维也纳, 2025 年 7 月。计算语言学协会。ISBN 979-8-89176-251-0。doi:10.18653/v1/2025.acl-long。

1598。 URL <https://aclanthology.org/2025.acl-long.1598/>。

H. Xia, T. Ge, P. Wang, S.-Q. Chen, F. Wei 和 Z. Sui。 “推测性解码:利用推测性执行加速序列到序列的生成”。载于 H. Bouamor, J. Pino 和 K. Bali 编辑的《计算语言学协会研究成果:EMNLP 2023》,第 3909-3925 页,新加坡, 2023 年 12 月。计算语言学协会。doi:10.18653/v1/2023.findings-emnlp.257。网址: <https://aclanthology.org/2023.findings-emnlp.257/>。

H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li 和 Z. Sui。 “释放大型语言模型推理的效率:推测解码的综合调查”。载于 L.-W. Ku, A. Martins 和 V. Srikumar 编辑的《计算语言学协会 (ACL) 2024 年会论文集》,第 7655-7671 页,泰国曼谷及线上会议, 2024 年 8 月。

计算语言学协会。doi:10.18653/v1/2024.findings-acl.456。URL: <https://aclanthology.org/2024.findings-acl.456>。

H. Xia, Y. Li, J. Zhang, C. Du 和 W. Li。 “SWIFT:用于 LLM 推理加速的即时自推测解码”。第十三届国际学习表征会议, 2025 年。网址: <https://openreview.net/forum?id=EKJhH5D5wA>。

Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, M. Xu, K. Yu, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang 和 W. Liang。 mHC:流形约束超连接。载于 2026 年第 43 届国际机器学习会议。网址: <https://openreview.net/forum?id=mDhyxu8WRB>。

T. Xu, E. Helenowski, K. A. Sankararaman, D. Jin, K. Peng, E. Han, S. Nie, C. Zhu, H. Zhang, W. Zhou 等。完美融合:利用混合评判者重新定义 rlhf。arXiv预印本 arXiv:2409.20370, 2024。

D. Yan, W. Wang 和 X. Chu。“揭秘张量核心以优化半精度矩阵乘法”。2020年 IEEE 国际并行与分布式处理研讨会 (IPDPS), 第 634–643 页, 2020 年。URL: <https://api.semanticscholar.org/CorpusID:220604999>。

A. 杨, A. 李, B. 杨, B. 张, B. 惠, B. 郑, B. 于, C. 高, C. 黄, C. 吕, 等。Q文3 技术报告。arXiv 预印本 arXiv:2505.09388, 2025。

Zacks917. AutoMTP_vLLM:将 vllm 适配到 automtp (多标记预测的提前停止)。https://github.com/Zacks917/AutoMTP_vLLM 2026年。访问日期:2026年6月21日。

J. Zhang, J. Wang, H. Li, L. Shou, K. Chen, G. Chen 和 S. Mehrotra。“Draft&verify:基于自推测解码的无损大型语言模型加速”。载于第 62 届计算语言学协会年会论文集 (第 1 卷:长篇论文), 第 11263–11282 页。计算语言学协会, 2024 年。doi: [10.18653/v1/2024.acl-long.607](https://doi.org/10.18653/v1/2024.acl-long.607)。URL: <http://dx.doi.org/10.18653/v1/2024.acl-long.607>。

J. Zhang, Z. Yu, S. Liu, E. J. Yu, Z. Li, D. Zhu, J. Duo, W. Xiong, Y. Song, G. Yu, J. Zhu 和 S. Li。Dflare:扩展块扩散推测解码的草稿容量, 2026a。URL <https://arxiv.org/abs/2606.02091>。

L. Zhang, X. Wang, Y. Huang 和 R. Xu。学习推测的协调表示 抽样, 2025 年。URL <https://arxiv.org/abs/2408.15766>。

S. Zhang, Y. Zhang, Z. Zhu, H. Wang, D. Ma, D. Zhang, L. Chen 和 K. Yu。Pacer:具有自适应长度的推测性解码的分块预验证。arXiv预印本 arXiv:2602.01274, 2026b。

Y. Zhang 和 T. Math-AI。美国邀请数学考试 (AIME) 2025, 2025。

C. Zhao, C. Deng, C. Ruan, D. Dai, H. Gao, J. Li, L. Zhang, P. Huang, S. Zhou, S. Ma 等。对 deepseek-v3 的深入研究:扩展性挑战和对人工智能架构硬件的思考。载于第 52 届计算机体系结构国际研讨会论文集, 第 1731–1745 页, 2025a。

W. 赵, T. 潘, X. 韩, Y. 张, S. Ao, Y. 黄, K. 张, W. 赵, Y. Li, J. Zhou, 等。Fr-spec:通过频率排序推测采样加速大词汇量语言模型。在计算语言学协会第 63 届年会论文集 (第 1 卷:长篇论文)中, 第 3909–3921 页, 2025b。

K. Zheng, Y. Chen, H. Mao, M.-Y. Liu, J. Zhu 和 Q. Zhang。掩码扩散模型是秘密的、与时间无关的掩码模型, 并利用不准确的类别采样。第十三届国际学习表征会议, 2025 年。网址: <https://openreview.net/forum?id=CTC7CmirNr>。

L. 郑, W. -L. 蒋, Y. 盛, 庄 S. 庄, Z. 吴, Y. 庄, Z. Lin, Z. Li, D. Li, E. Xing, 等。使用 mt-bench 和 chatbot arena 评估 llm 作为裁判的能力。神经信息处理系统进展, 36:46595–46623, 2023。

L. 郑, L. 尹, Z. 谢, C. 孙, J. 黄, CH Yu, S. 曹, C. Kozyrakis, I. Stoica, JE Gonzalez, C. Barrett 和 Y. Sheng。Sglang:结构化语言模型的高效执行

程序。见 A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak 和 C. Zhang 编辑的《神经信息处理系统进展》第 37 卷,第 62557–62583 页。Curran Associates, Inc., 2024 年。doi:10.52202/079017-2000。URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/724be4472168f31ba1c9ac630f15dec8-Paper-Conference.pdf。

Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin 和 H. Zhang。 “DistServe:面向高吞吐量优化的大型语言模型服务的预填充和解码分离”。第 18 届 USENIX 操作系统设计与实现研讨会 (OSDI 24),第 193–210 页, 2024 年。

朱坤,高勇,赵勇,赵立,左刚,谷勇,谢东,唐涛,徐强,叶正,K. Kamahori,C. -Y. Lin,Z. Wang,S. Wang,A. Krishnamurthy 和 B. Kasikci。 Nanoflow:实现服务吞吐量的最佳大型语言模型。第 19 届 USENIX 操作系统设计与实现会议论文集, OSDI 25,美国,2025 年。USENIX 协会。

ISBN 978-1-939133-47-2。

附录

A. 反例:不提前终止的选择偏差

我们提供一个简单的反例来说明离线全局搜索（即操作）是如何进行的。

如果算法1中缺少中断条件,则会违反所需的非预期特性。

通过无损推测解码。形式上,第 i 个草稿标记的准入事件, $\ell \geq i$ 必须由在标记被采样之前可获得的调度器可见信息确定。

它不应依赖于自身的实现。考虑一个只有单个请求的场景

($\ell = 1$)和最大选秀长度 ($\ell = 2$)。假设第一个位置的预代币置信度为

$1 = 0.8$,且轮廓容量曲线为

$$\text{SPS}(1) = 1.0, \quad \text{SPS}(2) = 0.5, \quad \text{SPS}(3) = 0.45。$$

验证 0 个和 1 个草稿代币的预期吞吐量为

$$\Theta_0 = 1 \cdot \text{SPS}(1) = 1.0,$$

$$\Theta_1 = (1 + 0.8) \cdot \text{SPS}(2) = 0.9。$$

如果不提前停止,调度器会在提交任何准入请求之前评估 Θ_2 。

决策。由于马尔可夫置信头使用了先前采样的令牌,因此下一个置信度得分 Θ_2 明确依赖于 1 的实现。因此,第二个前缀

生存概率

$$\theta_2 = 1.2$$

还取决于1.

考虑 1 的两种可能实现方式:

情况 1 (1产生较高的 θ_2):假设1的结果是 $\theta_2 = 0.9$ 。那么

$$\theta_2 = 0.8 \times 0.9 = 0.72。$$

长度为 2 的预期吞吐量为

$$\Theta_2 = (1 + 0.8 + 0.72) \times 0.45 = 1.134。$$

由于 Θ_2 是 $\{1.0, 0.9, 1.134\}$ 中的全局最大值,调度器返回 $\ell = 2$ 。

第一个令牌1被允许进入验证前缀。

情况 2 (1产生较低的 θ_2):假设1导致 $\theta_2 = 0$ 。那么

$$\theta_2 = 0。$$

长度为 2 的预期吞吐量为

$$\Theta_2 = (1 + 0.8 + 0) \times 0.45 = 0.81。$$

此处,全局最大值仍为 $\Theta_0 = 1.0$,因此调度器返回 $\ell = 0$ 。第一个标记

1不被允许进入验证前缀。

因此,首轮草案代币的准入动态取决于首轮草案的价值。

令牌本身。这种追溯性依赖会引入选择偏差:调度器更倾向于使用令牌。

即使录取决定应该做出,也会导致高度自信的后续行动。

在观察 1 之前已经做出。现在我们明确地说明分布偏差。令

词汇表为 $\{t, \bar{t}\}$,并考虑第一个位置的目标和草稿分布:

$$t() = 0.7, \quad \bar{t}() = 0.3,$$

$$d(1) = 0.5, \quad d(0) = 0.5。$$

第一位的标准投机性接受概率为

$$\sum_{\ell \in \{1,0\}} \text{分钟} t(\ell, d(\ell)) = \text{分钟}(0.7, 0.5) + \text{分钟}(0.3, 0.5) = 0.8,$$

与假定值 ($1 = 0.8$)相符。假设回顾性调度器的行为如上所述:

$1 =$ 产生较高的延续置信度,因此 $\ell = 2$,而 $1 =$ 产生低延续置信度,因此 $\ell = 0$ 。那么第一个输出标记的分布如下。如果 $1 =$, 草稿代币以一定的概率被接受。

$$1 \text{分钟}, \quad \frac{t(\ell)}{d(\ell)} = \min 1, 0.5 \frac{0.7}{0.5} = 1,$$

所以输出标记是。如 $1 =$, 草稿代币不被接受;取而代之的是目标模型。果从中生成一个新的令牌 t 。因此,

$$\Pr(1) = \Pr(1=) \cdot 1 + \Pr(1=) \cdot t(\ell) = 0.5 + 0.5 \times 0.7 = 0.85,$$

因此

$$\Pr(0) = 0.15。$$

该输出分布((0.85, 0.15))与目标分布((0.7, 0.3)) 不同,证明回顾性调度器并非无损的。提前停止机制可以防止这种情况发生。

因果贪婪调度器存在问题。由于 $\Theta 1 < \Theta 0$,调度器立即停止并返回。

在评估任何与延续相关的量 (例如 $\ell 2$)之前, $\ell = 0$ 。录取决定

因此,第一个位置仅取决于标记前的信息,不会受到偏见的影响。

通过实现这一点,可以恢复标准所要求的非预期属性。

无损论证。